



VIT-AP UNIVERSITY

CINEMATCH - MOVIE RECOMMENDATION AND ANALYTICS SYSTEM

A NoSQL Database Case Study Using MongoDB

Registration Number	Name
24MIC7198	V. RUTHVIK
24MIC7181	K. TEJASH NARAYANA
24MIC7203	B. HARSHITH

Under The Guidance Of : Dr. VASAVI SANIKOMMU

School of Computer Science and Engineering (SCOPE)

VIT-AP University, Amaravati

Problem Statement

Movie streaming platforms collect a large number of ratings from their users. This rating data is useful to find out which movies are popular, which genres are liked by the users, how active the users are and what movies a user may like to watch next. The MovieLens dataset contains this kind of data in four separate CSV files (movies, ratings, tags and links). The genre information of a movie is stored as a single string separated by the pipe symbol, and the ratings of a movie are stored in a different file, so answering simple questions like "what is the average rating of each genre" needs the files to be combined and processed.

In this project we use MongoDB, a document based NoSQL database, to store the MovieLens data and analyse it. The data is imported as collections, cleaned inside MongoDB, and then analysed using find queries and aggregation pipelines. The project also builds a simple recommendation feature using the same queries and shows the results in a web application.

Aim

The aim of this project is to store the MovieLens movie rating dataset in MongoDB and to analyse it using MongoDB queries and aggregation pipelines. The project focuses on finding information about movies, genres, ratings, users and tags, and on recommending movies to users using the rating data.

Objectives

1. To import the MovieLens CSV files into MongoDB as separate collections.
2. To clean the imported data inside MongoDB (extract the release year, split the genres string into an array and convert timestamps to dates).
3. To perform basic CRUD operations and queries using filters, projection, sorting, regular expressions and text search.
4. To analyse the ratings: rating distribution, most rated movies, highest and lowest rated movies and most controversial movies.
5. To analyse genres by joining the ratings and movies collections using \$lookup and \$unwind.
6. To analyse users: most active users, harsh and generous users, and rating activity over the years.
7. To analyse the tags given by users.
8. To recommend movies using genre similarity, "users who liked this also liked" and weighted rating.
9. To create indexes and check their effect using explain().
10. To display the results in a web application connected to MongoDB.

Dataset Description

The project uses the MovieLens "ml-latest-small" dataset published by GroupLens Research, University of Minnesota. The same dataset is available on Kaggle as "Popular Movies Datasets - 9742 Movies". It contains 100,836 ratings and 3,683 tags given by 610 users to 9,742 movies between 1996 and 2018. Every user has rated at least 20 movies. The dataset has four CSV files:

File	Rows	Columns	Description
movies.csv	9,742	movieId, title, genres	Movie title with the release year in brackets and genres separated by
ratings.csv	100,836	userId, movieId, rating, timestamp	Rating given by a user to a movie (0.5 to 5.0 stars) with Unix timestamp
tags.csv	3,683	userId, movieId, tag, timestamp	Free text tag given by a user to a movie
links.csv	9,742	movieId, imdbId, tmdbId	IMDb and TMDb ids of each movie

Sample rows from the files:

```
movies.csv
movieId,title,genres
1,Toy Story (1995),Adventure|Animation|Children|Comedy|Fantasy
2,Jumanji (1995),Adventure|Children|Fantasy
```

```
ratings.csv
userId,movieId,rating,timestamp
1,1,4.0,964982703
1,3,4.0,964981247
```

```
tags.csv
userId,movieId,tag,timestamp
2,60756,funny,1445714994
```

```
links.csv
movieId,imdbId,tmdbId
1,0114709,862
```

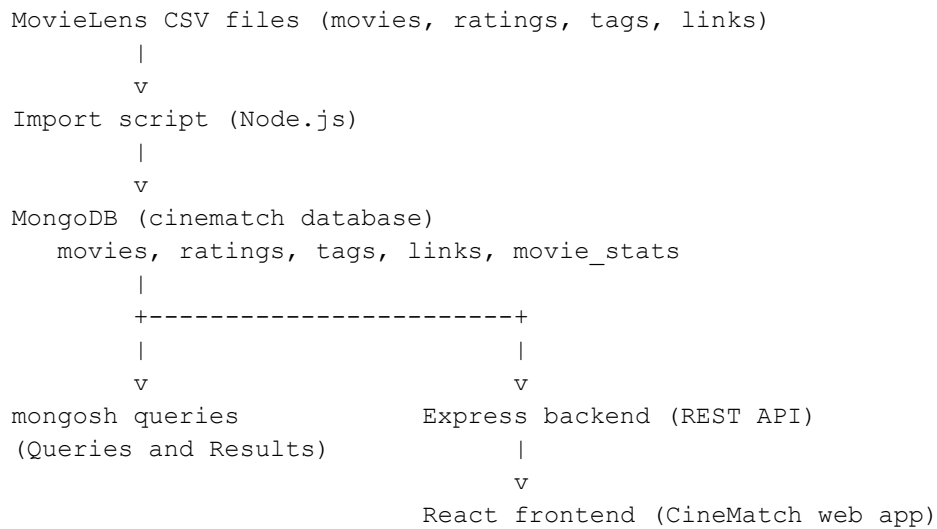
Why MongoDB

- A movie has many genres. In MongoDB this is stored as an array inside the movie document, so there is no need for a separate genre table and join.
- The data could be cleaned inside the database itself using updateMany with pipeline stages, without writing a separate program.
- The aggregation pipeline (\$match, \$group, \$sort, \$lookup, \$unwind, \$facet etc.) can answer complex questions in a single query.
- Ratings refer to movies through movieId (references), while genres are embedded in the movie document. Both NoSQL data modelling styles are used.
- Indexes on movieId, userId, the genre array and the title keep the queries fast, which was checked with explain().

Technology Stack

Part	Technology
Database	MongoDB Community Server 8.3
Database Shell	mongosh
Data Import	Node.js script using csv-parse and the MongoDB Node.js driver
Backend	Node.js, Express
Frontend	React (Vite), Recharts
Code Editor	Visual Studio Code

System Architecture



The CSV files are imported into four collections. Cleaning queries add the year, genre array and date fields. The same queries are run in mongosh for this report and are also used by the Express backend, which sends the results to the React frontend.

MongoDB Database Design

Database name: cinematch

Collection	Documents	Description
movies	9,742	Movie details. Fields: movieId, title, genres, year, genreList
ratings	100,836	Ratings given by users. Fields: userId, movieId, rating, timestamp, ratedAt
tags	3,683	Tags given by users. Fields: userId, movieId, tag, timestamp, taggedAt
links	9,742	IMDb and TMDb ids. Fields: movieId, imdbId, tmdbId
movie_stats	9,724	Rating count, average and standard deviation of every movie (created with \$merge)

year, genreList, ratedAt and taggedAt are new fields added by the cleaning queries.

Relationships

- ratings.movieId, tags.movieId and links.movieId refer to movies.movieId. They are joined using \$lookup.
- movie_stats._id is the movieId of the movie.
- There is no users collection because the dataset does not have user details. User information is calculated from the ratings collection.

Ratings are kept in a separate collection because a popular movie has hundreds of ratings and ratings are also queried by user and by date. Genres are embedded in the movie document as an array because they are small and always needed along with the movie.

Sample documents after cleaning

```
movies
{
  _id: ObjectId('...'),
```

```

    movieId: 1,
    title: 'Toy Story (1995)',
    genres: 'Adventure|Animation|Children|Comedy|Fantasy',
    year: 1995,
    genreList: [ 'Adventure', 'Animation', 'Children', 'Comedy', 'Fantasy' ]
  }

ratings
{ userId: 1, movieId: 1, rating: 4, timestamp: 964982703,
  ratedAt: ISODate('2000-07-30T18:45:03.000Z') }

tags
{ userId: 2, movieId: 60756, tag: 'funny', timestamp: 1445714994,
  taggedAt: ISODate('2015-10-24T19:29:54.000Z') }

movie_stats
{ _id: 356, ratingCount: 329, avgRating: 4.164, stdDev: 0.829 }

```

Indexes

Collection	Index	Purpose
ratings	{ movieId: 1 }	Joining ratings with movies
ratings	{ userId: 1 }	Queries by user
tags	{ movieId: 1 }	Tags of a movie
movies	{ movieId: 1 } (unique)	Primary lookup key
movies	{ genreList: 1 } (multikey)	Filtering by genre
movies	{ year: 1 }	Filtering by year
movies	{ title: 'text' }	Text search on titles

Implementation

In this project we used MongoDB as the NoSQL database to store and analyse the MovieLens data. The four CSV files were imported into the cinematch database as separate collections using a Node.js import script, which works the same way as mongoimport with the --headerline option. After importing, the collections and sample documents were checked in mongosh.

The imported data was then cleaned inside MongoDB. The release year was taken out of the title using \$regexFind, the genres string was split into an array using \$split, titles ending with ", The" were rewritten in the normal order, and the Unix timestamps were converted to dates using \$toDate. Indexes were created on the fields used for joining and filtering.

After this, different MongoDB operations were used to analyse the data. Basic find queries with filters, projection, sorting, regular expressions and text search were used to look up movies. Aggregation pipelines with \$group, \$lookup, \$unwind, \$sort, \$facet, \$bucket and date operators were used to analyse ratings, genres, users and tags. Recommendation queries were written using \$setIntersection (genre similarity), a \$lookup on the fans of a movie (users who liked this also liked) and a weighted rating formula. \$merge was used to store the per-movie statistics in a separate collection. Finally, CRUD operations and explain() were used to show insert, update, delete and the use of indexes.

All the queries were executed in mongosh and the output of each query is shown as a screenshot in the next section. The same queries are used by the backend of the CineMatch web application, which is shown at the end.

Queries and Results

Database Overview

1. List all collections in the cinematch database

Query:

```
db.getCollectionNames()
```

Output:

A screenshot of a terminal window titled "mongosh - cinematch". The prompt is "cinematch>". The command "db.getCollectionNames()" has been entered, and the output is "['ratings', 'movie_stats', 'movies', 'links', 'tags']". The prompt "cinematch>" is shown again on the next line.

```
mongosh - cinematch
cinematch> db.getCollectionNames()
[ 'ratings', 'movie_stats', 'movies', 'links', 'tags' ]
cinematch>
```

Result: The four collections created from the CSV files are listed (movie_stats is added later by Query 53).

2. Count documents in every collection

Query:

```
{
  movies: db.movies.countDocuments(),
  ratings: db.ratings.countDocuments(),
  tags: db.tags.countDocuments(),
  links: db.links.countDocuments()
}
```

Output:

A screenshot of a terminal window titled "mongosh - cinematch". The prompt is "cinematch>". The command "{ movies: db.movies.countDocuments(), ratings: db.ratings.countDocuments(), tags: db.tags.countDocuments(), links: db.links.countDocuments() }" has been entered. The output is "{ movies: 9742, ratings: 100836, tags: 3683, links: 9742 }". The prompt "cinematch>" is shown again on the next line.

```
mongosh - cinematch
cinematch> {
...   movies: db.movies.countDocuments(),
...   ratings: db.ratings.countDocuments(),
...   tags: db.tags.countDocuments(),
...   links: db.links.countDocuments()
... }
{ movies: 9742, ratings: 100836, tags: 3683, links: 9742 }
cinematch>
```

Result: The document counts match the dataset: 9,742 movies, 100,836 ratings, 3,683 tags and 9,742 links.

3. Display a sample movie document

Query:

```
db.movies.findOne({ movieId: 1 })
```

Output:

```
mongosh - cinematch

cinematch> db.movies.findOne({ movieId: 1 })

{
  _id: ObjectId('6aa6fbf4f05594b31fa64e5c'),
  movieId: 1,
  title: 'Toy Story (1995)',
  genres: 'Adventure|Animation|Children|Comedy|Fantasy'
}

cinematch> 
```

Result: The genres of the movie are stored as a single string separated by the pipe symbol.

4. Display a sample rating document

Query:

```
db.ratings.findOne({ userId: 1, movieId: 1 })
```

Output:

```
mongosh - cinematch

cinematch> db.ratings.findOne({ userId: 1, movieId: 1 })

{
  _id: ObjectId('6aa6fbf5f05594b31fa6746a'),
  userId: 1,
  movieId: 1,
  rating: 4,
  timestamp: 964982703
}

cinematch> 
```

Result: rating and timestamp are stored as numbers. The timestamp is in Unix seconds.

5. Display a sample tag and link document

Query:

```
[ db.tags.findOne({ movieId: 60756 }), db.links.findOne({ movieId: 1 }) ]
```

Output:

```
mongosh - cinematch

cinematch> [ db.tags.findOne({ movieId: 60756 }), db.links.findOne({ movieId: 1 }) ]

[
  {
    _id: ObjectId('6aa6fbf6f05594b31fa7fe4e'),
    userId: 2,
    movieId: 60756,
    tag: 'funny',
    timestamp: 1445714994
  },
  {
    _id: ObjectId('6aa6fbf6f05594b31fa80cb1'),
    movieId: 1,
    imdbId: '0114709',
    tmdbId: 862
  }
]

cinematch> 
```

Result: Tags are free text. imdbId is stored as a string so that the leading zero is kept.

6. Count the number of distinct users who rated movies

Query:

```
db.ratings.aggregate([{$group: { _id: "$userId" } }, { $count: "totalUsers"
}])
```

Output:

 mongosh - cinematch

— □ □

```
cinematch> db.ratings.aggregate([{$group: { _id: "$userId" } }, { $count: "totalUsers" }])
[ { totalUsers: 610 } ]
cinematch> █
```

Result: There are 610 users in the dataset.

Data Cleaning & Schema Enrichment

7. Extract the release year from the movie title

Query:

```
db.movies.updateMany({}, [
  {
    $set: {
      year: {
        $let: {
          vars: { m: { $regexFind: { input: "$title", regex:
/\\((\\d{4})\\)\\s*$ / } } },
          in: {
            $cond: [
              { $eq: ["$$m", null] },
              null,
              { $toInt: { $arrayElemAt: ["$$m.captures", 0] } }
            ]
          }
        }
      }
    }
  }
])
```

Output:

```
mongosh - cinematch

cinematch> db.movies.updateMany({}, [
...   {
...     $set: {
...       year: {
...         $let: {
...           vars: { m: { $regexFind: { input: "$title", regex: /\((\d{4})\)\s*$/ } } },
...           in: {
...             $cond: [
...               { $eq: ["$m", null] },
...               null,
...               { $toInt: { $arrayElemAt: ["$m.captures", 0] } }
...             ]
...           }
...         }
...       }
...     }
...   }
... ])

{
  acknowledged: true,
  insertedId: null,
  matchedCount: 9742,
  modifiedCount: 9742,
  upsertedCount: 0
}

cinematch>
```

Result: All 9,742 movie documents were updated with a numeric year field.

8. Split the pipe-delimited genres string into an array

Query:

```
db.movies.updateMany({}, [
  { $set: { genreList: { $split: ["$genres", "|"] } } }
])
```

Output:

```
mongosh - cinematch

cinematch> db.movies.updateMany({}, [
...   { $set: { genreList: { $split: ["$genres", "|"] } } }
...   ])

{
  acknowledged: true,
  insertedId: null,
  matchedCount: 9742,
  modifiedCount: 9742,
  upsertedCount: 0
}

cinematch>
```

Result: genreList is now an array, so the genres can be filtered and unwound individually.

9. Convert Unix timestamps into ISODate fields

Query:

```
db.ratings.updateMany({}, [
  { $set: { ratedAt: { $toDate: { $multiply: ["$timestamp", 1000] } } } }
]);
db.tags.updateMany({}, [
```

```

    { $set: { taggedAt: { $toDate: { $multiply: ["$timestamp", 1000] } } } } }
  ])
}

```

Output:

mongosh - cinematch

```

cinematch> db.ratings.updateMany({}, [
...   { $set: { ratedAt: { $toDate: { $multiply: ["$timestamp", 1000] } } } }
...   ]);
...   db.tags.updateMany({}, [
...     { $set: { taggedAt: { $toDate: { $multiply: ["$timestamp", 1000] } } } }
...     ])
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3683,
  modifiedCount: 3683,
  upsertedCount: 0
}
cinematch>

```

Result: All ratings and tags now have an ISODate field which can be used with \$year and \$dayOfWeek.

10. Normalise titles that end with a trailing article

Query:

```

db.movies.updateMany(
  { title: /, (The|A|An) \(/ },
  [
    {
      $set: {
        title: {
          $let: {
            vars: { m: { $regexFind: { input: "$title", regex: /^(.+),
(The|A|An) (\(.+)\$/ } } },
            in: {
              $concat: [
                { $arrayElemAt: ["$$m.captures", 1] }, " ",
                { $arrayElemAt: ["$$m.captures", 0] }, " ",
                { $arrayElemAt: ["$$m.captures", 2] }
              ]
            }
          }
        }
      }
    }
  ]
)

```

Output:

```
mongosh - cinematch

cinematch> db.movies.updateMany(
...   { title: /, (The|A|An) \(</> },
...   [
...     {
...       $set: {
...         title: {
...           $let: {
...             vars: { m: { $regexFind: { input: "$title", regex: /^(.+), (The|A|An) (\(<.+>$/ } } },
...             in: {
...               $concat: [
...                 { $arrayElemAt: [ "$$.captures", 1 ] }, " ",
...                 { $arrayElemAt: [ "$$.captures", 0 ] }, " ",
...                 { $arrayElemAt: [ "$$.captures", 2 ] }
...               ]
...             }
...           }
...         }
...       }
...     ]
...   ]
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1822,
  modifiedCount: 1822,
  upsertedCount: 0
}
cinematch> 
```

Result: 1,822 titles were rewritten, for example 'Shawshank Redemption, The (1994)' became 'The Shawshank Redemption (1994)'.

11. Verify the enriched movie documents

Query:

```
[ db.movies.findOne({ movieId: 1 }), db.movies.findOne({ movieId: 318 }, {
  _id: 0, title: 1, year: 1, genreList: 1 }) ]
```

Output:

```
mongosh - cinematch

cinematch> [ db.movies.findOne({ movieId: 1 }), db.movies.findOne({ movieId: 318 }, { _id: 0, title: 1, year: 1, genreList:
...   }) ]
[
  {
    _id: ObjectId('6aa6fbf4f05594b31fa64e5c'),
    movieId: 1,
    title: 'Toy Story (1995)',
    genres: 'Adventure|Animation|Children|Comedy|Fantasy',
    year: 1995,
    genreList: [ 'Adventure', 'Animation', 'Children', 'Comedy', 'Fantasy' ]
  },
  {
    title: 'The Shawshank Redemption (1994)',
    year: 1994,
    genreList: [ 'Crime', 'Drama' ]
  }
]
cinematch> 
```

Result: The Toy Story document now has year and genreList, and the title of movie 318 is in the normal order.

12. Create indexes on frequently queried fields

Query:

```
db.ratings.createIndex({ movieId: 1 });
db.ratings.createIndex({ userId: 1 });
db.tags.createIndex({ movieId: 1 });
db.movies.createIndex({ movieId: 1 }, { unique: true });
db.movies.createIndex({ genreList: 1 });
db.movies.createIndex({ year: 1 });
db.movies.createIndex({ title: "text" });
db.movies.getIndexes().map(i => i.name)
```

Output:



```
mongosh - cinematch

cinematch> db.ratings.createIndex({ movieId: 1 });
...
db.ratings.createIndex({ userId: 1 });
...
db.tags.createIndex({ movieId: 1 });
...
db.movies.createIndex({ movieId: 1 }, { unique: true });
...
db.movies.createIndex({ genreList: 1 });
...
db.movies.createIndex({ year: 1 });
...
db.movies.createIndex({ title: "text" });
...
db.movies.getIndexes().map(i => i.name)

[ '_id_', 'movieId_1', 'genreList_1', 'year_1', 'title_text' ]

cinematch>
```

Result: The indexes on the movies collection are listed, including the multikey index on genreList and the text index on title.

Movie Exploration

13. Find movies released in 1995

Query:

```
db.movies.find({ year: 1995 }, { _id: 0, movieId: 1, title: 1, genres: 1
}).limit(10)
```

Output:

mongosh - cinematch

```
cinematch> db.movies.find({ year: 1995 }, { _id: 0, movieId: 1, title: 1, genres: 1 }).limit(10)

[
  {
    movieId: 1,
    title: 'Toy Story (1995)',
    genres: 'Adventure|Animation|Children|Comedy|Fantasy'
  },
  {
    movieId: 2,
    title: 'Jumanji (1995)',
    genres: 'Adventure|Children|Fantasy'
  },
  {
    movieId: 3,
    title: 'Grumpier Old Men (1995)',
    genres: 'Comedy|Romance'
  },
  {
    movieId: 4,
    title: 'Waiting to Exhale (1995)',
    genres: 'Comedy|Drama|Romance'
  },
  {
    movieId: 5,
    title: 'Father of the Bride Part II (1995)',
    genres: 'Comedy'
  },
  { movieId: 6, title: 'Heat (1995)', genres: 'Action|Crime|Thriller' },
  { movieId: 7, title: 'Sabrina (1995)', genres: 'Comedy|Romance' },
  {
    movieId: 8,
    title: 'Tom and Huck (1995)',
    genres: 'Adventure|Children'
  },
  { movieId: 9, title: 'Sudden Death (1995)', genres: 'Action' },
  {
    movieId: 10,
    title: 'GoldenEye (1995)',
    genres: 'Action|Adventure|Thriller'
  }
]

cinematch> 
```

Result: There are 259 movies released in 1995. Only the first 10 are shown.

14. Search movie titles with a regular expression

Query:

```
db.movies.find({ title: { $regex: "star wars", $options: "i" } }, { _id: 0,
title: 1, year: 1 }).sort({ year: 1 })
```

Output:

mongosh - cinematch

```
cinematch> db.movies.find({ title: { $regex: "star wars", $options: "i" } }, { _id: 0, title: 1, year: 1 }).sort({ year: 1 })
[
  { title: 'Star Wars: Episode IV - A New Hope (1977)', year: 1977 },
  { title: 'The Star Wars Holiday Special (1978)', year: 1978 },
  {
    title: 'Star Wars: Episode V - The Empire Strikes Back (1980)',
    year: 1980
  },
  {
    title: 'Star Wars: Episode VI - Return of the Jedi (1983)',
    year: 1983
  },
  {
    title: 'Star Wars: Episode I - The Phantom Menace (1999)',
    year: 1999
  },
  {
    title: 'Star Wars: Episode II - Attack of the Clones (2002)',
    year: 2002
  },
  {
    title: 'Empire of Dreams: The Story of the 'Star Wars' Trilogy (2004)',
    year: 2004
  },
  {
    title: 'Star Wars: Episode III - Revenge of the Sith (2005)',
    year: 2005
  },
  { title: 'Star Wars: The Clone Wars (2008)', year: 2008 },
  {
    title: 'Star Wars: Episode VII - The Force Awakens (2015)',
    year: 2015
  },
  { title: 'Rogue One: A Star Wars Story (2016)', year: 2016 },
  { title: 'Star Wars: The Last Jedi (2017)', year: 2017 },
  { title: 'Solo: A Star Wars Story (2018)', year: 2018 }
]
cinematch> 
```

Result: 13 movies contain 'star wars' in the title. They are shown in order of year.

15. Find movies that are both Comedy and Romance

Query:

```
db.movies.find({ genreList: { $all: ["Comedy", "Romance"] }, year: { $gte: 2015 } }, { _id: 0, title: 1, genres: 1 }).limit(10)
```

Output:

```
mongosh - cinematch

cinematch> db.movies.find({ genreList: { $all: ["Comedy", "Romance"] }, year: { $gte: 2015 }, { _id: 0, title: 1, genres: 1 } }).limit(10)

[
  {
    title: 'Mortdecai (2015)',
    genres: 'Adventure|Comedy|Mystery|Romance'
  },
  { title: 'Boy Meets Girl (2015)', genres: 'Comedy|Drama|Romance' },
  { title: 'Focus (2015)', genres: 'Comedy|Crime|Drama|Romance' },
  { title: 'Man Up (2015)', genres: 'Comedy|Romance' },
  { title: 'The Lobster (2015)', genres: 'Comedy|Romance|Sci-Fi' },
  { title: 'Trainwreck (2015)', genres: 'Comedy|Romance' },
  { title: 'Aloha (2015)', genres: 'Comedy|Drama|Romance' },
  { title: 'The Escort (2015)', genres: 'Comedy|Romance' },
  { title: 'Slow Learners (2015)', genres: 'Comedy|Romance' },
  { title: 'How to Be Single (2016)', genres: 'Comedy|Romance' }
]

cinematch>
```

Result: Only movies having both Comedy and Romance in the genre array and released in 2015 or later are returned.

16. Count movies with no genres listed

Query:

```
db.movies.countDocuments({ genres: "(no genres listed)" })
```

Output:

```
mongosh - cinematch

cinematch> db.movies.countDocuments({ genres: "(no genres listed)" })

34

cinematch>
```

Result: 34 movies have no genres listed.

17. Number of movies per genre

Query:

```
db.movies.aggregate([
  { $unwind: "$genreList" },
  { $group: { _id: "$genreList", movieCount: { $sum: 1 } } },
  { $sort: { movieCount: -1 } }
])
```

Output:

```
mongosh - cinematch

cinematch> db.movies.aggregate([
...     { $unwind: "$genreList" },
...     { $group: { _id: "$genreList", movieCount: { $sum: 1 } } },
...     { $sort: { movieCount: -1 } }
... ])

[
  { _id: 'Drama', movieCount: 4361 },
  { _id: 'Comedy', movieCount: 3756 },
  { _id: 'Thriller', movieCount: 1894 },
  { _id: 'Action', movieCount: 1828 },
  { _id: 'Romance', movieCount: 1596 },
  { _id: 'Adventure', movieCount: 1263 },
  { _id: 'Crime', movieCount: 1199 },
  { _id: 'Sci-Fi', movieCount: 980 },
  { _id: 'Horror', movieCount: 978 },
  { _id: 'Fantasy', movieCount: 779 },
  { _id: 'Children', movieCount: 664 },
  { _id: 'Animation', movieCount: 611 },
  { _id: 'Mystery', movieCount: 573 },
  { _id: 'Documentary', movieCount: 440 },
  { _id: 'War', movieCount: 382 },
  { _id: 'Musical', movieCount: 334 },
  { _id: 'Western', movieCount: 167 },
  { _id: 'IMAX', movieCount: 158 },
  { _id: 'Film-Noir', movieCount: 87 },
  { _id: '(no genres listed)', movieCount: 34 }
]

cinematch>
```

Result: Drama (4,361) and Comedy (3,756) have the most movies. Film-Noir (87) has the least.

18. Number of movies released per decade

Query:

```
db.movies.aggregate([
  { $match: { year: { $ne: null } } },
  {
    $group: {
      _id: { $multiply: [{ $floor: { $divide: ["$year", 10] } }, 10] },
      movies: { $sum: 1 }
    }
  },
  { $sort: { _id: 1 } }
])
```

Output:

```
mongosh - cinematch

cinematch> db.movies.aggregate([
...   { $match: { year: { $ne: null } } },
...   {
...     $group: {
...       _id: { $multiply: [{ $floor: { $divide: ["$year", 10] } }, 10] },
...       movies: { $sum: 1 }
...     }
...   },
...   { $sort: { _id: 1 } }
... ])

[
  { _id: 1900, movies: 3 },
  { _id: 1910, movies: 7 },
  { _id: 1920, movies: 37 },
  { _id: 1930, movies: 136 },
  { _id: 1940, movies: 197 },
  { _id: 1950, movies: 279 },
  { _id: 1960, movies: 401 },
  { _id: 1970, movies: 500 },
  { _id: 1980, movies: 1177 },
  { _id: 1990, movies: 2212 },
  { _id: 2000, movies: 2849 },
  { _id: 2010, movies: 1931 }
]

cinematch> 
```

Result: The 1990s and 2000s have more than 2,000 movies each. Very few movies are before 1950.

19. Top 10 years with the most movie releases

Query:

```
db.movies.aggregate([
  { $match: { year: { $ne: null } } },
  { $group: { _id: "$year", movies: { $sum: 1 } } },
  { $sort: { movies: -1 } },
  { $limit: 10 }
])
```

Output:

```
mongosh - cinematch

cinematch> db.movies.aggregate([
...   { $match: { year: { $ne: null } } },
...   { $group: { _id: "$year", movies: { $sum: 1 } } },
...   { $sort: { movies: -1 } },
...   { $limit: 10 }
... ])

[
  { _id: 2002, movies: 311 },
  { _id: 2006, movies: 295 },
  { _id: 2001, movies: 294 },
  { _id: 2007, movies: 284 },
  { _id: 2000, movies: 283 },
  { _id: 2009, movies: 282 },
  { _id: 2004, movies: 279 },
  { _id: 2003, movies: 279 },
  { _id: 2014, movies: 278 },
  { _id: 1996, movies: 276 }
]

cinematch> 
```

Result: 2002 has the most movies (311), followed by 2006 (295) and 2001 (294).

20. Most common genre combinations

Query:

```
db.movies.aggregate([
  { $match: { genreList: { $not: { $size: 1 } } } },
  { $group: { _id: "$genres", count: { $sum: 1 } } },
  { $sort: { count: -1 } },
  { $limit: 10 }
])
```

Output:

```
mongosh - cinematch

cinematch> db.movies.aggregate([
...   { $match: { genreList: { $not: { $size: 1 } } } },
...   { $group: { _id: "$genres", count: { $sum: 1 } } },
...   { $sort: { count: -1 } },
...   { $limit: 10 }
... ])
[
  { _id: 'Comedy|Drama', count: 435 },
  { _id: 'Comedy|Romance', count: 363 },
  { _id: 'Drama|Romance', count: 349 },
  { _id: 'Comedy|Drama|Romance', count: 276 },
  { _id: 'Drama|Thriller', count: 168 },
  { _id: 'Horror|Thriller', count: 135 },
  { _id: 'Crime|Drama', count: 134 },
  { _id: 'Crime|Drama|Thriller', count: 125 },
  { _id: 'Drama|War', count: 114 },
  { _id: 'Comedy|Crime', count: 101 }
]
cinematch>
```

Result: Comedy|Drama (435), Comedy|Romance (363) and Drama|Romance (349) are the most common genre combinations.

21. Full-text search on movie titles

Query:

```
db.movies.find({ $text: { $search: "lord rings" } }, { _id: 0, title: 1,
score: { $meta: "textScore" } }).sort({ score: { $meta: "textScore" }
}).limit(5)
```

Output:

```
mongosh - cinematch

cinematch> db.movies.find({ $text: { $search: "lord rings" } }, { _id: 0, title: 1, score: { $meta: "textScore" } }).sort({
...   score: { $meta: "textScore" } }).limit(5)
[
  {
    title: 'The Lord of the Rings: The Fellowship of the Ring (2001)',
    score: 1.65
  },
  { title: 'The Lord of the Rings (1978)', score: 1.3333333333333333 },
  {
    title: 'The Lord of the Rings: The Return of the King (2003)',
    score: 1.2
  },
  { title: 'The Lord of the Rings: The Two Towers (2002)', score: 1.2 },
  { title: 'The Ring (2002)', score: 0.75 }
]
cinematch>
```

Result: The text search returns the Lord of the Rings movies first, with their text score.

Rating Analysis

22. Rating distribution (how often each star value is used)

Query:

```
db.ratings.aggregate([
  { $group: { _id: "$rating", count: { $sum: 1 } } },
  { $sort: { _id: 1 } }
])
```

Output:

```
mongosh - cinematch

cinematch> db.ratings.aggregate([
...     { $group: { _id: "$rating", count: { $sum: 1 } } },
...     { $sort: { _id: 1 } }
... ])

[
  { _id: 0.5, count: 1370 },
  { _id: 1, count: 2811 },
  { _id: 1.5, count: 1791 },
  { _id: 2, count: 7551 },
  { _id: 2.5, count: 5550 },
  { _id: 3, count: 20047 },
  { _id: 3.5, count: 13136 },
  { _id: 4, count: 26818 },
  { _id: 4.5, count: 8551 },
  { _id: 5, count: 13211 }
]

cinematch>
```

Result: 4.0 is the most common rating (26,818 times), followed by 3.0 and 5.0. Whole star ratings are used more than half star ratings.

23. Overall rating statistics

Query:

```
db.ratings.aggregate([
  {
    $group: {
      _id: null,
      totalRatings: { $sum: 1 },
      avgRating: { $avg: "$rating" },
      minRating: { $min: "$rating" },
      maxRating: { $max: "$rating" },
      stdDev: { $stdDevPop: "$rating" }
    }
  },
  {
    $project: {
      _id: 0,
      totalRatings: 1,
      avgRating: { $round: ["$avgRating", 3] },
      minRating: 1,
      maxRating: 1,
      stdDev: { $round: ["$stdDev", 3] }
    }
  }
])
```

Output:

```
mongosh - cinematch

cinematch> db.ratings.aggregate([
...   {
...     $group: {
...       _id: null,
...       totalRatings: { $sum: 1 },
...       avgRating: { $avg: "$rating" },
...       minRating: { $min: "$rating" },
...       maxRating: { $max: "$rating" },
...       stdDev: { $stdDevPop: "$rating" }
...     }
...   },
...   {
...     $project: {
...       _id: 0,
...       totalRatings: 1,
...       avgRating: { $round: ["$avgRating", 3] },
...       minRating: 1,
...       maxRating: 1,
...       stdDev: { $round: ["$stdDev", 3] }
...     }
...   }
... ])

[
  {
    totalRatings: 100836,
    minRating: 0.5,
    maxRating: 5,
    avgRating: 3.502,
    stdDev: 1.043
  }
]

cinematch> 
```

Result: The overall average rating is 3.50 with a standard deviation of 1.04.

24. Average rating of a specific movie (Toy Story)

Query:

```
db.ratings.aggregate([
  { $match: { movieId: 1 } },
  {
    $group: {
      _id: "$movieId",
      ratings: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  }
])
```

Output:

```
mongosh - cinematch

cinematch> db.ratings.aggregate([
...   { $match: { movieId: 1 } },
...   {
...     $group: {
...       _id: "$movieId",
...       ratings: { $sum: 1 },
...       avgRating: { $avg: "$rating" }
...     }
...   }
... ])

[ { _id: 1, ratings: 215, avgRating: 3.9209302325581397 } ]

cinematch>
```

Result: Toy Story has 215 ratings with an average of 3.92.

25. Top 10 most-rated (most popular) movies

Query:

```
db.ratings.aggregate([
  {
    $group: {
      _id: "$movieId",
      ratingCount: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },
  { $sort: { ratingCount: -1 } },
  { $limit: 10 },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  { $unwind: "$movie" },
  {
    $project: {
      _id: 0,
      title: "$movie.title",
      ratingCount: 1,
      avgRating: { $round: ["$avgRating", 2] }
    }
  }
])
```

Output:

```
mongosh - cinematch

cinematch> db.ratings.aggregate([
...   {
...     $group: {
...       _id: "$movieId",
...       ratingCount: { $sum: 1 },
...       avgRating: { $avg: "$rating" }
...     },
...   },
...   { $sort: { ratingCount: -1 } },
...   { $limit: 10 },
...   {
...     $lookup: {
...       from: "movies",
...       localField: "_id",
...       foreignField: "movieId",
...       as: "movie"
...     }
...   },
...   { $unwind: "$movie" },
...   {
...     $project: {
...       _id: 0,
...       title: "$movie.title",
...       ratingCount: 1,
...       avgRating: { $round: ["$avgRating", 2] }
...     }
...   }
... ])

[
  { ratingCount: 329, title: 'Forrest Gump (1994)', avgRating: 4.16 },
  {
    ratingCount: 317,
    title: 'The Shawshank Redemption (1994)',
    avgRating: 4.43
  },
  { ratingCount: 307, title: 'Pulp Fiction (1994)', avgRating: 4.2 },
  {
    ratingCount: 279,
    title: 'The Silence of the Lambs (1991)',
    avgRating: 4.16
  },
  { ratingCount: 278, title: 'The Matrix (1999)', avgRating: 4.19 },
  {
    ratingCount: 251,
    title: 'Star Wars: Episode IV - A New Hope (1977)',
    avgRating: 4.23
  },
  { ratingCount: 238, title: 'Jurassic Park (1993)', avgRating: 3.75 },
  { ratingCount: 237, title: 'Braveheart (1995)', avgRating: 4.03 },
  {
    ratingCount: 224,
    title: 'Terminator 2: Judgment Day (1991)',
    avgRating: 3.97
  },
  {
    ratingCount: 220,
    title: 'Schindler's List (1993)',
    avgRating: 4.22
  }
]

cinematch> 
```

Result: Forrest Gump (329 ratings), The Shawshank Redemption (317) and Pulp Fiction (307) are the most rated movies.

26. Top 10 highest-rated movies (minimum 100 ratings)

Query:

```
db.ratings.aggregate([
  {
    $group: {
      _id: "$movieId",
      ratingCount: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },
  { $match: { ratingCount: { $gte: 100 } } },
  { $sort: { avgRating: -1 } },
  { $limit: 10 },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  { $unwind: "$movie" },
  {
    $project: {
      _id: 0,
      title: "$movie.title",
      year: "$movie.year",
      ratingCount: 1,
      avgRating: { $round: ["$avgRating", 2] }
    }
  }
])
```

Output:

```
mongosh - cinematch

cinematch> db.ratings.aggregate([
...   {
...     $group: {
...       _id: "$movieId",
...       ratingCount: { $sum: 1 },
...       avgRating: { $avg: "$rating" }
...     }
...   },
...   { $match: { ratingCount: { $gte: 100 } } },
...   { $sort: { avgRating: -1 } },
...   { $limit: 10 },
...   {
...     $lookup: {
...       from: "movies",
...       localField: "_id",
...       foreignField: "movieId",
...       as: "movie"
...     }
...   },
...   { $unwind: "$movie" },
...   {
...     $project: {
...       _id: 0,
...       title: "$movie.title",
...       year: "$movie.year",
...       ratingCount: 1,
...       avgRating: { $round: ["$avgRating", 2] }
...     }
...   }
... ])

[
  {
    ratingCount: 317,
    title: 'The Shawshank Redemption (1994)',
    year: 1994,
    avgRating: 4.43
  },
  {
    ratingCount: 192,
    title: 'The Godfather (1972)',
    year: 1972,
    avgRating: 4.29
  },
  {
    ratingCount: 218,
    title: 'Fight Club (1999)',
    year: 1999,
    avgRating: 4.27
  },
  {
    ratingCount: 129,
    title: 'The Godfather: Part II (1974)',
    year: 1974,
    avgRating: 4.26
  },
  {
    ratingCount: 107,
    title: 'The Departed (2006)',
    year: 2006,
    avgRating: 4.25
  },
  {
    ratingCount: 126,
    title: 'Goodfellas (1990)',
    year: 1990,
    avgRating: 4.25
  },
  {
    ratingCount: 100,
    title: 'Casablanca (1942)',
    year: 1942,
    avgRating: 4.24
  },
  {
    ratingCount: 149,
    title: 'The Dark Knight (2008)',
    year: 2008,
    avgRating: 4.24
  },
  {
    ratingCount: 204,
    title: 'The Usual Suspects (1995)',
    year: 1995,
    avgRating: 4.24
  },
  {
    ratingCount: 142,
    title: 'The Princess Bride (1987)',
    year: 1987,
    avgRating: 4.23
  }
]

cinematch> █
```

Result: Among movies with at least 100 ratings, The Shawshank Redemption (4.43) has the highest average, followed by The Godfather (4.29) and Fight Club (4.27).

27. 10 lowest-rated movies (minimum 30 ratings)

Query:

```
db.ratings.aggregate([
  {
    $group: {
      _id: "$movieId",
      ratingCount: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },
  { $match: { ratingCount: { $gte: 30 } } },
  { $sort: { avgRating: 1 } },
  { $limit: 10 },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  { $unwind: "$movie" },
  {
    $project: {
      _id: 0,
      title: "$movie.title",
      ratingCount: 1,
      avgRating: { $round: ["$avgRating", 2] }
    }
  }
])
```

Output:

mongosh - cinematch

```
cinematch> db.ratings.aggregate([
...   {
...     $group: {
...       _id: "$movieId",
...       ratingCount: { $sum: 1 },
...       avgRating: { $avg: "$rating" }
...     },
...   },
...   { $match: { ratingCount: { $gte: 30 } } },
...   { $sort: { avgRating: 1 } },
...   { $limit: 10 },
...   {
...     $lookup: {
...       from: "movies",
...       localField: "_id",
...       foreignField: "movieId",
...       as: "movie"
...     }
...   },
...   { $unwind: "$movie" },
...   {
...     $project: {
...       _id: 0,
...       title: "$movie.title",
...       ratingCount: 1,
...       avgRating: { $round: ["$avgRating", 2] }
...     }
...   }
... ])

[
  { ratingCount: 33, title: 'Godzilla (1998)', avgRating: 1.95 },
  {
    ratingCount: 32,
    title: 'I Know What You Did Last Summer (1997)',
    avgRating: 2.11
  },
  { ratingCount: 53, title: 'Wild Wild West (1999)', avgRating: 2.21 },
  { ratingCount: 42, title: 'Batman & Robin (1997)', avgRating: 2.21 },
  { ratingCount: 39, title: 'Hollow Man (2000)', avgRating: 2.29 },
  { ratingCount: 42, title: 'The Flintstones (1994)', avgRating: 2.36 },
  { ratingCount: 37, title: 'Free Willy (1993)', avgRating: 2.39 },
  { ratingCount: 41, title: 'Striptease (1996)', avgRating: 2.41 },
  { ratingCount: 63, title: 'Coneheads (1993)', avgRating: 2.42 },
  { ratingCount: 35, title: 'Showgirls (1995)', avgRating: 2.47 }
]

cinematch> 
```

Result: Godzilla (1998) has the lowest average (1.95), followed by I Know What You Did Last Summer, Wild Wild West and Batman & Robin.

28. Most controversial movies (highest rating variance)

Query:

```
db.ratings.aggregate([
  {
    $group: {
      _id: "$movieId",
      ratingCount: { $sum: 1 },
      avgRating: { $avg: "$rating" },
      stdDev: { $stdDevPop: "$rating" }
    }
  },
  { $match: { ratingCount: { $gte: 50 } } },
  { $sort: { stdDev: -1 } },
])
```

```
{ $limit: 10 },
{
  $lookup: {
    from: "movies",
    localField: "_id",
    foreignField: "movieId",
    as: "movie"
  }
},
{ $unwind: "$movie" },
{
  $project: {
    _id: 0,
    title: "$movie.title",
    ratingCount: 1,
    avgRating: { $round: ["$avgRating", 2] },
    stdDev: { $round: ["$stdDev", 3] }
  }
}
])
```

Output:

```
mongosh - cinematch

cinematch> db.ratings.aggregate([
...   {
...     $group: {
...       _id: "$movieId",
...       ratingCount: { $sum: 1 },
...       avgRating: { $avg: "$rating" },
...       stdDev: { $stdDevPop: "$rating" }
...     }
...   },
...   { $match: { ratingCount: { $gte: 50 } } },
...   { $sort: { stdDev: -1 } },
...   { $limit: 10 },
...   {
...     $lookup: {
...       from: "movies",
...       localField: "_id",
...       foreignField: "movieId",
...       as: "movie"
...     }
...   },
...   { $unwind: "$movie" },
...   {
...     $project: {
...       _id: 0,
...       title: "$movie.title",
...       ratingCount: 1,
...       avgRating: { $round: ["$avgRating", 2] },
...       stdDev: { $round: ["$stdDev", 3] }
...     }
...   }
... ])

[
  {
    ratingCount: 64,
    title: 'The Blair Witch Project (1999)',
    avgRating: 2.8,
    stdDev: 1.363
  },
  {
    ratingCount: 54,
    title: 'First Knight (1995)',
    avgRating: 3.08,
    stdDev: 1.287
  },
  {
    ratingCount: 121,
    title: 'Austin Powers: The Spy Who Shagged Me (1999)',
    avgRating: 3.2,
    stdDev: 1.21
  },
  {
    ratingCount: 70,
    title: 'Scream (1996)',
    avgRating: 3.2,
    stdDev: 1.187
  },
  {
    ratingCount: 55,
    title: 'Moulin Rouge (2001)',
    avgRating: 3.59,
    stdDev: 1.187
  },
  {
    ratingCount: 86,
    title: 'Mars Attacks! (1996)',
    avgRating: 3.09,
    stdDev: 1.173
  },
  {
    ratingCount: 82,
    title: 'The Nutty Professor (1996)',
    avgRating: 2.73,
    stdDev: 1.167
  },
  {
    ratingCount: 54,
    title: 'The Cable Guy (1996)',
    avgRating: 2.81,
    stdDev: 1.164
  },
  {
    ratingCount: 50,
    title: 'Scary Movie (2000)',
    avgRating: 2.92,
    stdDev: 1.159
  },
  {
    ratingCount: 56,
    title: 'Rushmore (1998)',
    avgRating: 3.54,
    stdDev: 1.147
  }
]

cinematch> 
```

Result: The Blair Witch Project has the highest standard deviation (1.36), which means users strongly disagree about it.

29. Movies with the most 5-star ratings

Query:

```
db.ratings.aggregate([
  { $match: { rating: 5 } },
  { $group: { _id: "$movieId", fiveStars: { $sum: 1 } } },
  { $sort: { fiveStars: -1 } },
  { $limit: 10 },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  { $unwind: "$movie" },
  { $project: { _id: 0, title: "$movie.title", fiveStars: 1 } }
])
```

Output:

```
mongosh - cinematch

cinematch> db.ratings.aggregate([
...   { $match: { rating: 5 } },
...   { $group: { _id: "$movieId", fiveStars: { $sum: 1 } } },
...   { $sort: { fiveStars: -1 } },
...   { $limit: 10 },
...   {
...     $lookup: {
...       from: "movies",
...       localField: "_id",
...       foreignField: "movieId",
...       as: "movie"
...     }
...   },
...   { $unwind: "$movie" },
...   { $project: { _id: 0, title: "$movie.title", fiveStars: 1 } }
... ])

[
  { fiveStars: 153, title: 'The Shawshank Redemption (1994)' },
  { fiveStars: 123, title: 'Pulp Fiction (1994)' },
  { fiveStars: 116, title: 'Forrest Gump (1994)' },
  { fiveStars: 109, title: 'The Matrix (1999)' },
  {
    fiveStars: 104,
    title: 'Star Wars: Episode IV - A New Hope (1977)'
  },
  { fiveStars: 92, title: 'The Silence of the Lambs (1991)' },
  { fiveStars: 92, title: 'Schindler's List (1993)' },
  { fiveStars: 88, title: 'The Godfather (1972)' },
  { fiveStars: 81, title: 'Fight Club (1999)' },
  { fiveStars: 80, title: 'Braveheart (1995)' }
]

cinematch> 
```

Result: The Shawshank Redemption (153), Pulp Fiction (123) and Forrest Gump (116) have the most 5 star ratings.

30. Movies that have never been rated

Query:

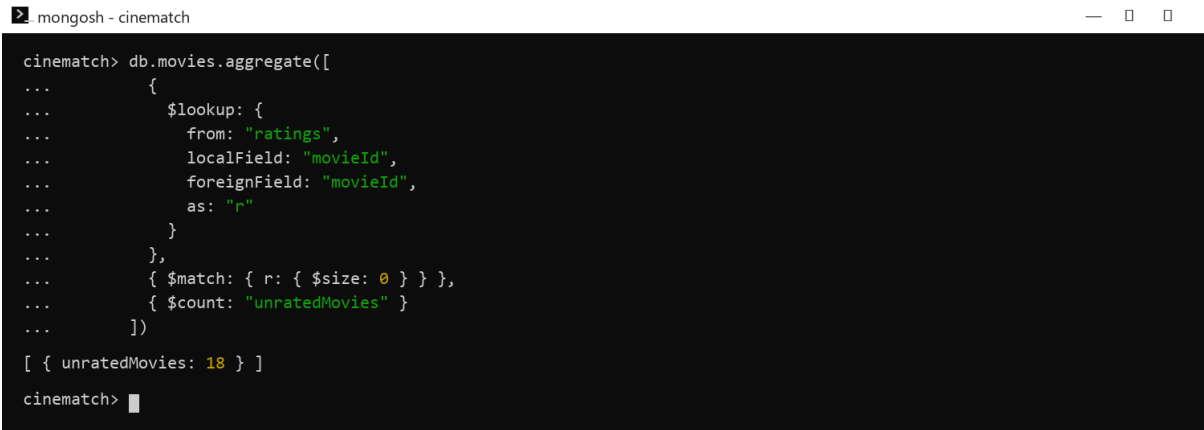
```
db.movies.aggregate([
  {
```

```

    $lookup: {
      from: "ratings",
      localField: "movieId",
      foreignField: "movieId",
      as: "r"
    }
  },
  { $match: { r: { $size: 0 } } },
  { $count: "unratedMovies" }
])

```

Output:



The screenshot shows a terminal window titled 'mongosh - cinematch'. The user enters an aggregation query on the 'movies' collection. The query uses a \$lookup stage to join with the 'ratings' collection, a \$match stage to filter for movies with no ratings (r.size: 0), and a \$count stage to count them as 'unratedMovies'. The result is a single document: [{ unratedMovies: 18 }].

```

cinematch> db.movies.aggregate([
...   {
...     $lookup: {
...       from: "ratings",
...       localField: "movieId",
...       foreignField: "movieId",
...       as: "r"
...     }
...   },
...   { $match: { r: { $size: 0 } } },
...   { $count: "unratedMovies" }
... ])
[ { unratedMovies: 18 } ]
cinematch>

```

Result: 18 movies in the movies collection have no ratings at all.

31. Weighted rating (IMDb-style Bayesian formula) – Top 10

Query:

```

db.ratings.aggregate([
  {
    $group: { _id: "$movieId", v: { $sum: 1 }, R: { $avg: "$rating" } }
  },
  { $match: { v: { $gte: 50 } } },
  {
    $addFields: {
      weighted: {
        $add: [
          {
            $multiply: [{ $divide: ["$v", { $add: ["$v", 50] }] }, "$R"]
          },
          {
            $multiply: [{ $divide: [50, { $add: ["$v", 50] }] }, 3.5]
          }
        ]
      }
    }
  },
  { $sort: { weighted: -1 } },
  { $limit: 10 },
  {
    $lookup: {
      from: "movies",
      localField: "_id",

```

```
        foreignField: "movieId",
        as: "movie"
    }
},
{ $unwind: "$movie" },
{
    $project: {
        _id: 0,
        title: "$movie.title",
        votes: "$v",
        avgRating: { $round: ["$R", 2] },
        weightedRating: { $round: ["$weighted", 3] }
    }
}
])
```

Output:

```
mongosh - cinematch

cinematch> db.ratings.aggregate([
...   {
...     $group: { _id: "$movieId", v: { $sum: 1 }, R: { $avg: "$rating" } }
...   },
...   { $match: { v: { $gte: 50 } } },
...   {
...     $addFields: {
...       weighted: {
...         $add: [
...           {
...             $multiply: [{ $divide: ["$v", { $add: ["$v", 50] } ] }, "$R" ]
...           },
...           {
...             $multiply: [{ $divide: [50, { $add: ["$v", 50] } ] }, 3.5 ]
...           }
...         ]
...       }
...     },
...     { $sort: { weighted: -1 } },
...     { $limit: 10 },
...     {
...       $lookup: {
...         from: "movies",
...         localField: "_id",
...         foreignField: "movieId",
...         as: "movie"
...       }
...     },
...     { $unwind: "$movie" },
...     {
...       $project: {
...         _id: 0,
...         title: "$movie.title",
...         votes: "$v",
...         avgRating: { $round: ["$R", 2] },
...         weightedRating: { $round: ["$weighted", 3] }
...       }
...     }
...   })
... ]
[
  {
    title: 'The Shawshank Redemption (1994)',
    votes: 317,
    avgRating: 4.43,
    weightedRating: 4.382
  },
  {
    title: 'Fight Club (1999)',
    votes: 218,
    avgRating: 4.27,
    weightedRating: 4.129
  },
  {
    title: 'The Godfather (1972)',
    votes: 192,
    avgRating: 4.29,
    weightedRating: 4.126
  },
  {
    title: 'Star Wars: Episode IV - A New Hope (1977)',
    votes: 251,
    avgRating: 4.23,
    weightedRating: 4.11
  },
  {
    title: 'Pulp Fiction (1994)',
    votes: 307,
    avgRating: 4.2,
    weightedRating: 4.099
  },
  {
    title: 'The Usual Suspects (1995)',
    votes: 204,
    avgRating: 4.24,
    weightedRating: 4.093
  },
  {
    title: 'Schindler's List (1993)',
    votes: 220,
    avgRating: 4.22,
    weightedRating: 4.091
  },
  {
    title: 'The Matrix (1999)',
    votes: 278,
    avgRating: 4.19,
    weightedRating: 4.087
  },
  {
    title: 'Star Wars: Episode V - The Empire Strikes Back (1980)',
    votes: 211,
    avgRating: 4.22,
    weightedRating: 4.079
  },
  {
    title: 'Forrest Gump (1994)',
    votes: 329,
    avgRating: 4.16,
    weightedRating: 4.077
  }
]
cinematch> 
```

Result: The weighted rating gives a balanced top 10 in which movies with very few ratings do not appear.

Genre Analysis

32. Average rating per genre

Query:

```
db.movies.aggregate([
  {
    $lookup: {
      from: "ratings",
      localField: "movieId",
      foreignField: "movieId",
      as: "r"
    }
  },
  { $unwind: "$r" },
  { $unwind: "$genreList" },
  {
    $group: {
      _id: "$genreList",
      avgRating: { $avg: "$r.rating" },
      ratings: { $sum: 1 }
    }
  },
  {
    $project: { avgRating: { $round: ["$avgRating", 3] }, ratings: 1 }
  },
  { $sort: { avgRating: -1 } }
])
```

Output:

mongosh - cinematch

```
cinematch> db.movies.aggregate([
...   {
...     $lookup: {
...       from: "ratings",
...       localField: "movieId",
...       foreignField: "movieId",
...       as: "r"
...     }
...   },
...   { $unwind: "$r" },
...   { $unwind: "$genreList" },
...   {
...     $group: {
...       _id: "$genreList",
...       avgRating: { $avg: "$r.rating" },
...       ratings: { $sum: 1 }
...     }
...   },
...   {
...     $project: { avgRating: { $round: ["$avgRating", 3] }, ratings: 1 }
...   },
...   { $sort: { avgRating: -1 } }
... ])

[
  { _id: 'Film-Noir', ratings: 870, avgRating: 3.92 },
  { _id: 'War', ratings: 4859, avgRating: 3.808 },
  { _id: 'Documentary', ratings: 1219, avgRating: 3.798 },
  { _id: 'Crime', ratings: 16681, avgRating: 3.658 },
  { _id: 'Drama', ratings: 41928, avgRating: 3.656 },
  { _id: 'Mystery', ratings: 7674, avgRating: 3.632 },
  { _id: 'Animation', ratings: 6988, avgRating: 3.63 },
  { _id: 'IMAX', ratings: 4145, avgRating: 3.618 },
  { _id: 'Western', ratings: 1930, avgRating: 3.584 },
  { _id: 'Musical', ratings: 4138, avgRating: 3.564 },
  { _id: 'Adventure', ratings: 24161, avgRating: 3.509 },
  { _id: 'Romance', ratings: 18124, avgRating: 3.507 },
  { _id: 'Thriller', ratings: 26452, avgRating: 3.494 },
  { _id: 'Fantasy', ratings: 11834, avgRating: 3.491 },
  { _id: '(no genres listed)', ratings: 47, avgRating: 3.489 },
  { _id: 'Sci-Fi', ratings: 17243, avgRating: 3.456 },
  { _id: 'Action', ratings: 30635, avgRating: 3.448 },
  { _id: 'Children', ratings: 9208, avgRating: 3.413 },
  { _id: 'Comedy', ratings: 39053, avgRating: 3.385 },
  { _id: 'Horror', ratings: 7291, avgRating: 3.258 }
]

cinematch> 
```

Result: Film-Noir (3.92), War (3.81) and Documentary (3.80) have the highest average rating. Horror (3.26) has the lowest.

33. Most popular genres by number of ratings

Query:

```
db.movies.aggregate([
  {
    $lookup: {
      from: "ratings",
      localField: "movieId",
      foreignField: "movieId",
      as: "r"
    }
  },
  { $project: { genreList: 1, n: { $size: "$r" } } },
  { $unwind: "$genreList" },
```

```

    { $group: { _id: "$genreList", ratings: { $sum: "$n" } } },
    { $sort: { ratings: -1 } },
    { $limit: 10 }
  ])
}

```

Output:

mongosh - cinematch

```

cinematch> db.movies.aggregate([
...   {
...     $lookup: {
...       from: "ratings",
...       localField: "movieId",
...       foreignField: "movieId",
...       as: "r"
...     }
...   },
...   { $project: { genreList: 1, n: { $size: "$r" } } },
...   { $unwind: "$genreList" },
...   { $group: { _id: "$genreList", ratings: { $sum: "$n" } } },
...   { $sort: { ratings: -1 } },
...   { $limit: 10 }
... ])
[
  { _id: 'Drama', ratings: 41928 },
  { _id: 'Comedy', ratings: 39053 },
  { _id: 'Action', ratings: 30635 },
  { _id: 'Thriller', ratings: 26452 },
  { _id: 'Adventure', ratings: 24161 },
  { _id: 'Romance', ratings: 18124 },
  { _id: 'Sci-Fi', ratings: 17243 },
  { _id: 'Crime', ratings: 16681 },
  { _id: 'Fantasy', ratings: 11834 },
  { _id: 'Children', ratings: 9208 }
]
cinematch>

```

Result: Drama (41,928 ratings) and Comedy (39,053) are the most watched genres.

34. Best movie in each genre (minimum 50 ratings)

Query:

```

db.ratings.aggregate([
  {
    $group: {
      _id: "$movieId",
      ratingCount: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },
  { $match: { ratingCount: { $gte: 50 } } },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  { $unwind: "$movie" },
  { $unwind: "$movie.genreList" },
  { $sort: { avgRating: -1 } },
  {

```

```
$group: {
  _id: "$movie.genreList",
  bestMovie: { $first: "$movie.title" },
  avgRating: { $first: { $round: ["$avgRating", 2] } },
  ratingCount: { $first: "$ratingCount" }
}
},
{ $sort: { _id: 1 } }
])
```

Output:

```
mongosh - cinematch
cinematch> db.ratings.aggregate([
...   {
...     $group: {
...       _id: "$movieId",
...       ratingCount: { $sum: 1 },
...       avgRating: { $avg: "$rating" }
...     },
...     $match: { ratingCount: { $gte: 50 } },
...     $lookup: {
...       from: "movies",
...       localField: "_id",
...       foreignField: "movieId",
...       as: "movie"
...     },
...     $unwind: "$movie",
...     $sort: { avgRating: -1 },
...     $group: {
...       _id: "$movie.genrelist",
...       bestMovie: { $first: "$movie.title" },
...       avgRating: { $first: { $round: [ "$avgRating", 2 ] } },
...       ratingCount: { $first: "$ratingCount" }
...     },
...     $sort: { _id: 1 }
...   })
... ]
[
  {
    _id: 'Action',
    bestMovie: 'Fight Club (1999)',
    avgRating: 4.27,
    ratingCount: 218
  },
  {
    _id: 'Adventure',
    bestMovie: 'The Princess Bride (1987)',
    avgRating: 4.23,
    ratingCount: 142
  },
  {
    _id: 'Animation',
    bestMovie: 'Spirited Away (Sen to Chihiro no kamikakushi) (2001)',
    avgRating: 4.16,
    ratingCount: 87
  },
  {
    _id: 'Children',
    bestMovie: 'Toy Story 3 (2010)',
    avgRating: 4.11,
    ratingCount: 55
  },
  {
    _id: 'Comedy',
    bestMovie: 'Dr. Strangelove or: How I Learned to Stop Worrying and Love the Bomb (1964)',
    avgRating: 4.27,
    ratingCount: 97
  },
  {
    _id: 'Crime',
    bestMovie: 'The Shawshank Redemption (1994)',
    avgRating: 4.43,
    ratingCount: 317
  },
  {
    _id: 'Documentary',
    bestMovie: 'Bowling for Columbine (2002)',
    avgRating: 3.78,
    ratingCount: 58
  },
  {
    _id: 'Drama',
    bestMovie: 'The Shawshank Redemption (1994)',
    avgRating: 4.43,
    ratingCount: 317
  },
  {
    _id: 'Fantasy',
    bestMovie: 'The Princess Bride (1987)',
    avgRating: 4.23,
    ratingCount: 142
  },
  {
    _id: 'Film-Noir',
    bestMovie: 'Chinatown (1974)',
    avgRating: 4.21,
    ratingCount: 59
  },
  {
    _id: 'Horror',
    bestMovie: 'The Silence of the Lambs (1991)',
    ... (52 more lines)
  }
]
cinematch>
```

Result: The best rated movie of each genre is shown, for example The Shawshank Redemption for Crime and Drama and Spirited Away for Animation.

35. Genre popularity by decade (Action vs Drama vs Comedy)

Query:

```
db.movies.aggregate([
  {
    $match: {
      year: { $gte: 1970 },
      genreList: { $in: ["Action", "Drama", "Comedy"] }
    }
  },
  { $unwind: "$genreList" },
  { $match: { genreList: { $in: ["Action", "Drama", "Comedy"] } } },
  {
    $group: {
      _id: {
        decade: { $multiply: [{ $floor: { $divide: ["$year", 10] } }, 10] },
        genre: "$genreList"
      },
      movies: { $sum: 1 }
    }
  },
  { $sort: { "_id.decade": 1, "_id.genre": 1 } }
])
```

Output:

mongosh - cinematch

— □ □

```
cinematch> db.movies.aggregate([
...   {
...     $match: {
...       year: { $gte: 1970 },
...       genreList: { $in: ["Action", "Drama", "Comedy"] }
...     }
...   },
...   { $unwind: "$genreList" },
...   { $match: { genreList: { $in: ["Action", "Drama", "Comedy"] } } },
...   {
...     $group: {
...       _id: {
...         decade: { $multiply: [{ $floor: { $divide: ["$year", 10] } }, 10] },
...         genre: "$genreList"
...       },
...       movies: { $sum: 1 }
...     }
...   },
...   { $sort: { "_id.decade": 1, "_id.genre": 1 } }
... ])

[
  { _id: { decade: 1970, genre: 'Action' }, movies: 103 },
  { _id: { decade: 1970, genre: 'Comedy' }, movies: 155 },
  { _id: { decade: 1970, genre: 'Drama' }, movies: 225 },
  { _id: { decade: 1980, genre: 'Action' }, movies: 227 },
  { _id: { decade: 1980, genre: 'Comedy' }, movies: 492 },
  { _id: { decade: 1980, genre: 'Drama' }, movies: 439 },
  { _id: { decade: 1990, genre: 'Action' }, movies: 392 },
  { _id: { decade: 1990, genre: 'Comedy' }, movies: 890 },
  { _id: { decade: 1990, genre: 'Drama' }, movies: 1038 },
  { _id: { decade: 2000, genre: 'Action' }, movies: 549 },
  { _id: { decade: 2000, genre: 'Comedy' }, movies: 1135 },
  { _id: { decade: 2000, genre: 'Drama' }, movies: 1365 },
  { _id: { decade: 2010, genre: 'Action' }, movies: 464 },
  { _id: { decade: 2010, genre: 'Comedy' }, movies: 758 },
  { _id: { decade: 2010, genre: 'Drama' }, movies: 743 }
]

cinematch> █
```

Result: Drama has more movies than Comedy and Action in every decade. All three genres have the most movies in the 2000s.

36. Average number of genres per movie

Query:

```
db.movies.aggregate([
  { $project: { genreCount: { $size: "$genreList" } } },
  {
    $group: {
      _id: null,
      avgGenres: { $avg: "$genreCount" },
      maxGenres: { $max: "$genreCount" }
    }
  },
  {
    $project: {
      _id: 0,
      avgGenres: { $round: ["$avgGenres", 2] },
      maxGenres: 1
    }
  }
])
```

Output:



```
mongosh - cinematch

cinematch> db.movies.aggregate([
...   { $project: { genreCount: { $size: "$genreList" } } },
...   {
...     $group: {
...       _id: null,
...       avgGenres: { $avg: "$genreCount" },
...       maxGenres: { $max: "$genreCount" }
...     }
...   },
...   {
...     $project: {
...       _id: 0,
...       avgGenres: { $round: ["$avgGenres", 2] },
...       maxGenres: 1
...     }
...   }
... ])
[ { maxGenres: 10, avgGenres: 2.27 } ]

cinematch>
```

Result: A movie has 2.27 genres on average and the maximum is 10.

User Analysis

37. Top 10 most active users

Query:

```
db.ratings.aggregate([
  {
    $group: {
      _id: "$userId",
      ratings: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  }
])
```

```

    }
  },
  { $sort: { ratings: -1 } },
  { $limit: 10 },
  {
    $project: {
      userId: "$_id",
      _id: 0,
      ratings: 1,
      avgRating: { $round: ["$avgRating", 2] }
    }
  }
]
])

```

Output:

```

mongosh - cinematch
cinematch> db.ratings.aggregate([
...   {
...     $group: {
...       _id: "$userId",
...       ratings: { $sum: 1 },
...       avgRating: { $avg: "$rating" }
...     }
...   },
...   { $sort: { ratings: -1 } },
...   { $limit: 10 },
...   {
...     $project: {
...       userId: "$_id",
...       _id: 0,
...       ratings: 1,
...       avgRating: { $round: ["$avgRating", 2] }
...     }
...   }
... ])
[
  { ratings: 2698, userId: 414, avgRating: 3.39 },
  { ratings: 2478, userId: 599, avgRating: 2.64 },
  { ratings: 2108, userId: 474, avgRating: 3.4 },
  { ratings: 1864, userId: 448, avgRating: 2.85 },
  { ratings: 1346, userId: 274, avgRating: 3.24 },
  { ratings: 1302, userId: 610, avgRating: 3.69 },
  { ratings: 1260, userId: 68, avgRating: 3.23 },
  { ratings: 1218, userId: 380, avgRating: 3.67 },
  { ratings: 1115, userId: 606, avgRating: 3.66 },
  { ratings: 1055, userId: 288, avgRating: 3.15 }
]
cinematch>

```

Result: User 414 has given the most ratings (2,698), followed by user 599 (2,478) and user 474 (2,108).

38. Harshest and most generous critics (minimum 100 ratings)

Query:

```

db.ratings.aggregate([
  {
    $group: {
      _id: "$userId",
      ratings: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },

```

```

{ $match: { ratings: { $gte: 100 } } },
{
  $project: {
    userId: "$_id",
    _id: 0,
    ratings: 1,
    avgRating: { $round: ["$avgRating", 2] }
  }
},
{
  $facet: {
    harshest: [{ $sort: { avgRating: 1 } }, { $limit: 5 }],
    mostGenerous: [{ $sort: { avgRating: -1 } }, { $limit: 5 } ]
  }
}
])

```

Output:

mongosh - cinematch

```

cinematch> db.ratings.aggregate([
...   {
...     $group: {
...       _id: "$userId",
...       ratings: { $sum: 1 },
...       avgRating: { $avg: "$rating" }
...     }
...   },
...   { $match: { ratings: { $gte: 100 } } },
...   {
...     $project: {
...       userId: "$_id",
...       _id: 0,
...       ratings: 1,
...       avgRating: { $round: ["$avgRating", 2] }
...     }
...   },
...   {
...     $facet: {
...       harshest: [{ $sort: { avgRating: 1 } }, { $limit: 5 }],
...       mostGenerous: [{ $sort: { avgRating: -1 } }, { $limit: 5 } ]
...     }
...   }
... ])
[
  {
    harshest: [
      { ratings: 194, userId: 139, avgRating: 2.14 },
      { ratings: 179, userId: 153, avgRating: 2.22 },
      { ratings: 385, userId: 567, avgRating: 2.25 },
      { ratings: 939, userId: 298, avgRating: 2.36 },
      { ratings: 400, userId: 517, avgRating: 2.39 }
    ],
    mostGenerous: [
      { ratings: 202, userId: 452, avgRating: 4.56 },
      { ratings: 292, userId: 122, avgRating: 4.55 },
      { ratings: 114, userId: 43, avgRating: 4.55 },
      { ratings: 130, userId: 52, avgRating: 4.48 },
      { ratings: 101, userId: 601, avgRating: 4.43 }
    ]
  }
]
cinematch>

```

Result: User 139 gives the lowest average rating (2.14) and user 452 gives the highest (4.56). Both lists are returned by one \$facet query.

39. Distribution of ratings per user

Query:

```
db.ratings.aggregate([
  { $group: { _id: "$userId", ratings: { $sum: 1 } } },
  {
    $bucket: {
      groupBy: "$ratings",
      boundaries: [20, 50, 100, 200, 500, 1000, 3000],
      default: "3000+",
      output: { users: { $sum: 1 } }
    }
  }
])
```

Output:



The screenshot shows a MongoDB terminal window titled 'mongosh - cinematch'. The user enters the following aggregation query:

```
cinematch> db.ratings.aggregate([
...   { $group: { _id: "$userId", ratings: { $sum: 1 } } },
...   {
...     $bucket: {
...       groupBy: "$ratings",
...       boundaries: [20, 50, 100, 200, 500, 1000, 3000],
...       default: "3000+",
...       output: { users: { $sum: 1 } }
...     }
...   }
... ])
```

The terminal displays the following output:

```
[
  { _id: 20, users: 225 },
  { _id: 50, users: 137 },
  { _id: 100, users: 114 },
  { _id: 200, users: 90 },
  { _id: 500, users: 32 },
  { _id: 1000, users: 12 }
]
```

The prompt 'cinematch>' is visible at the bottom of the terminal.

Result: Most users have between 20 and 99 ratings. Only 12 users have more than 1,000 ratings.

40. Favourite genres of a specific user (userId 1)

Query:

```
db.ratings.aggregate([
  { $match: { userId: 1 } },
  {
    $lookup: {
      from: "movies",
      localField: "movieId",
      foreignField: "movieId",
      as: "movie"
    }
  },
  { $unwind: "$movie" },
  { $unwind: "$movie.genreList" },
  {
    $group: {
      _id: "$movie.genreList",
      rated: { $sum: 1 },
    }
  }
])
```

```

        avgRating: { $avg: "$rating" }
    }
},
{
    $project: { rated: 1, avgRating: { $round: ["$avgRating", 2] } }
},
{ $sort: { rated: -1 } },
{ $limit: 5 }
])

```

Output:

```

mongosh - cinematch

cinematch> db.ratings.aggregate([
...   { $match: { userId: 1 } },
...   {
...     $lookup: {
...       from: "movies",
...       localField: "movieId",
...       foreignField: "movieId",
...       as: "movie"
...     }
...   },
...   { $unwind: "$movie" },
...   { $unwind: "$movie.genreList" },
...   {
...     $group: {
...       _id: "$movie.genreList",
...       rated: { $sum: 1 },
...       avgRating: { $avg: "$rating" }
...     }
...   },
...   {
...     $project: { rated: 1, avgRating: { $round: ["$avgRating", 2] } }
...   },
...   { $sort: { rated: -1 } },
...   { $limit: 5 }
... ])

[
  { _id: 'Action', rated: 90, avgRating: 4.32 },
  { _id: 'Adventure', rated: 85, avgRating: 4.39 },
  { _id: 'Comedy', rated: 83, avgRating: 4.28 },
  { _id: 'Drama', rated: 68, avgRating: 4.53 },
  { _id: 'Thriller', rated: 55, avgRating: 4.15 }
]

cinematch>

```

Result: User 1 has rated Action, Adventure and Comedy movies the most, and gives the highest average to Drama (4.53).

41. Number of ratings submitted per year

Query:

```

db.ratings.aggregate([
  {
    $group: {
      _id: { $year: "$ratedAt" },
      ratings: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },
  {
    $project: { ratings: 1, avgRating: { $round: ["$avgRating", 2] } }
  },

```

```

    { $sort: { _id: 1 } }
  ]).toArray()

```

Output:

mongosh - cinematch

```

cinematch> db.ratings.aggregate([
...   {
...     $group: {
...       _id: { $year: "$ratedAt" },
...       ratings: { $sum: 1 },
...       avgRating: { $avg: "$rating" }
...     }
...   },
...   {
...     $project: { ratings: 1, avgRating: { $round: ["$avgRating", 2] } }
...   },
...   { $sort: { _id: 1 } }
... ]).toArray()
[
  { _id: 1996, ratings: 6040, avgRating: 3.54 },
  { _id: 1997, ratings: 1916, avgRating: 3.73 },
  { _id: 1998, ratings: 507, avgRating: 3.44 },
  { _id: 1999, ratings: 2439, avgRating: 3.77 },
  { _id: 2000, ratings: 10061, avgRating: 3.39 },
  { _id: 2001, ratings: 3922, avgRating: 3.51 },
  { _id: 2002, ratings: 3478, avgRating: 3.61 },
  { _id: 2003, ratings: 4014, avgRating: 3.5 },
  { _id: 2004, ratings: 3279, avgRating: 3.51 },
  { _id: 2005, ratings: 5813, avgRating: 3.43 },
  { _id: 2006, ratings: 4059, avgRating: 3.49 },
  { _id: 2007, ratings: 7114, avgRating: 3.31 },
  { _id: 2008, ratings: 4351, avgRating: 3.66 },
  { _id: 2009, ratings: 4158, avgRating: 3.39 },
  { _id: 2010, ratings: 2301, avgRating: 3.46 },
  { _id: 2011, ratings: 1690, avgRating: 3.78 },
  { _id: 2012, ratings: 4656, avgRating: 3.64 },
  { _id: 2013, ratings: 1664, avgRating: 3.88 },
  { _id: 2014, ratings: 1439, avgRating: 3.51 },
  { _id: 2015, ratings: 6616, avgRating: 3.51 },
  { _id: 2016, ratings: 6703, avgRating: 3.69 },
  { _id: 2017, ratings: 8198, avgRating: 3.35 },
  { _id: 2018, ratings: 6418, avgRating: 3.39 }
]
cinematch>

```

Result: The most ratings were given in the year 2000 (10,061), followed by 2007 and 2015.

42. Ratings by day of the week

Query:

```

db.ratings.aggregate([
  {
    $group: { _id: { $dayOfWeek: "$ratedAt" }, ratings: { $sum: 1 } }
  },
  { $sort: { _id: 1 } },
  {
    $project: {
      _id: 0,
      day: {
        $arrayElemAt: [
          ["", "Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"],
          "$_id"
        ]
      }
    },
    ratings: 1
  }
])

```

```

    }
  }
])

```

Output:

mongosh - cinematch

```

cinematch> db.ratings.aggregate([
...   {
...     $group: { _id: { $dayOfWeek: "$ratedAt" }, ratings: { $sum: 1 } }
...   },
...   { $sort: { _id: 1 } },
...   {
...     $project: {
...       _id: 0,
...       day: {
...         $arrayElemAt: [
...           [ "", "Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"],
...           "$_id"
...         ]
...       },
...       ratings: 1
...     }
...   }
... ])
[
  { ratings: 16209, day: 'Sun' },
  { ratings: 17583, day: 'Mon' },
  { ratings: 16411, day: 'Tue' },
  { ratings: 14014, day: 'Wed' },
  { ratings: 10706, day: 'Thu' },
  { ratings: 14455, day: 'Fri' },
  { ratings: 11458, day: 'Sat' }
]
cinematch>

```

Result: Monday has the most ratings (17,583) and Thursday has the least (10,706).

43. First and last rating date of the most active user

Query:

```

db.ratings.aggregate([
  { $match: { userId: 414 } },
  {
    $group: {
      _id: "$userId",
      firstRating: { $min: "$ratedAt" },
      lastRating: { $max: "$ratedAt" },
      total: { $sum: 1 }
    }
  }
])

```

Output:

```
mongosh - cinematch

cinematch> db.ratings.aggregate([
...   { $match: { userId: 414 } },
...   {
...     $group: {
...       _id: "$userId",
...       firstRating: { $min: "$ratedAt" },
...       lastRating: { $max: "$ratedAt" },
...       total: { $sum: 1 }
...     }
...   }
... ])

[
  {
    _id: 414,
    firstRating: ISODate('2000-06-19T17:36:56.000Z'),
    lastRating: ISODate('2018-06-02T22:21:12.000Z'),
    total: 2698
  }
]

cinematch>
```

Result: User 414 gave ratings from June 2000 to June 2018.

Tag Analysis

44. Most frequently used tags

Query:

```
db.tags.aggregate([
  { $group: { _id: { $toLower: "$tag" }, count: { $sum: 1 } } },
  { $sort: { count: -1 } },
  { $limit: 15 }
])
```

Output:

```
mongosh - cinematch

cinematch> db.tags.aggregate([
...   { $group: { _id: { $toLower: "$tag" }, count: { $sum: 1 } } },
...   { $sort: { count: -1 } },
...   { $limit: 15 }
... ])

[
  { _id: 'in netflix queue', count: 131 },
  { _id: 'atmospheric', count: 41 },
  { _id: 'surreal', count: 24 },
  { _id: 'thought-provoking', count: 24 },
  { _id: 'funny', count: 24 },
  { _id: 'superhero', count: 24 },
  { _id: 'disney', count: 23 },
  { _id: 'sci-fi', count: 23 },
  { _id: 'religion', count: 22 },
  { _id: 'quirky', count: 22 },
  { _id: 'psychology', count: 21 },
  { _id: 'dark comedy', count: 21 },
  { _id: 'suspense', count: 21 },
  { _id: 'visually appealing', count: 20 },
  { _id: 'twist ending', count: 20 }
]

cinematch>
```

Result: 'in netflix queue' is the most used tag (131 times), followed by 'atmospheric'.

45. All tags applied to Pulp Fiction

Query:

```
db.tags.aggregate([
  { $match: { movieId: 296 } },
  {
    $group: {
      _id: "$movieId",
      tags: { $addToSet: "$tag" },
      taggers: { $addToSet: "$userId" }
    }
  },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  {
    $project: {
      _id: 0,
      title: { $first: "$movie.title" },
      totalTags: { $size: "$tags" },
      taggerCount: { $size: "$taggers" },
      sampleTags: { $slice: ["$tags", 15] }
    }
  }
])
```

Output:

```
mongosh - cinematch

cinematch> db.tags.aggregate([
...   { $match: { movieId: 296 } },
...   {
...     $group: {
...       _id: "$movieId",
...       tags: { $addToSet: "$tag" },
...       taggers: { $addToSet: "$userId" }
...     },
...     {
...       $lookup: {
...         from: "movies",
...         localField: "_id",
...         foreignField: "movieId",
...         as: "movie"
...       }
...     },
...     {
...       $project: {
...         _id: 0,
...         title: { $first: "$movie.title" },
...         totalTags: { $size: "$tags" },
...         taggerCount: { $size: "$taggers" },
...         sampleTags: { $slice: ["$tags", 15] }
...       }
...     }
...   })
[
  {
    title: 'Pulp Fiction (1994)',
    totalTags: 173,
    taggerCount: 4,
    sampleTags: [
      'Palme d'Or',
      'r:strong bloody violence',
      'stylish',
      'mobster',
      'characters',
      'drugs',
      'great acting',
      'rape',
      'film noir',
      'r:graphic sexuality',
      'masterpiece',
      'coke',
      'nonlinear',
      'dark humor',
      'exciting'
    ]
  }
]
cinematch> 
```

Result: Pulp Fiction has 173 different tags given by 4 users.

46. Most tagged movies

Query:

```
db.tags.aggregate([
  { $group: { _id: "$movieId", tagCount: { $sum: 1 } } },
  { $sort: { tagCount: -1 } },
  { $limit: 10 },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
```

```

        foreignField: "movieId",
        as: "movie"
    }
},
{ $unwind: "$movie" },
{ $project: { _id: 0, title: "$movie.title", tagCount: 1 } }
])

```

Output:

mongosh - cinematch

```

cinematch> db.tags.aggregate([
...   { $group: { _id: "$movieId", tagCount: { $sum: 1 } } },
...   { $sort: { tagCount: -1 } },
...   { $limit: 10 },
...   {
...     $lookup: {
...       from: "movies",
...       localField: "_id",
...       foreignField: "movieId",
...       as: "movie"
...     }
...   },
...   { $unwind: "$movie" },
...   { $project: { _id: 0, title: "$movie.title", tagCount: 1 } }
... ])

[
  { tagCount: 181, title: 'Pulp Fiction (1994)' },
  { tagCount: 54, title: 'Fight Club (1999)' },
  { tagCount: 41, title: '2001: A Space Odyssey (1968)' },
  {
    tagCount: 35,
    title: 'Léon: The Professional (a.k.a. The Professional) (Léon) (1994)'
  },
  {
    tagCount: 34,
    title: 'Eternal Sunshine of the Spotless Mind (2004)'
  },
  { tagCount: 32, title: 'The Big Lebowski (1998)' },
  { tagCount: 29, title: 'Donnie Darko (2001)' },
  { tagCount: 26, title: 'Inception (2010)' },
  { tagCount: 26, title: 'Star Wars: Episode IV - A New Hope (1977)' },
  { tagCount: 19, title: 'Suicide Squad (2016)' }
]

cinematch>

```

Result: Pulp Fiction (181) and Fight Club (54) are the most tagged movies.

47. Average rating of movies tagged 'funny'

Query:

```

db.tags.aggregate([
  { $match: { tag: { $regex: "^funny$", $options: "i" } } },
  { $group: { _id: "$movieId" } },
  {
    $lookup: {
      from: "ratings",
      localField: "_id",
      foreignField: "movieId",
      as: "r"
    }
  },
  {
    $lookup: {
      from: "movies",

```

```
        localField: "_id",
        foreignField: "movieId",
        as: "movie"
    }
},
{
    $project: {
        _id: 0,
        title: { $first: "$movie.title" },
        ratings: { $size: "$r" },
        avgRating: { $round: [{ $avg: "$r.rating" }, 2] }
    }
},
{ $sort: { ratings: -1 } },
{ $limit: 10 }
])
```

Output:

mongosh - cinematch

```
cinematch> db.tags.aggregate([
...   { $match: { tag: { $regex: "^funny$", $options: "i" } } },
...   { $group: { _id: "$movieId" } },
...   {
...     $lookup: {
...       from: "ratings",
...       localField: "_id",
...       foreignField: "movieId",
...       as: "r"
...     }
...   },
...   {
...     $lookup: {
...       from: "movies",
...       localField: "_id",
...       foreignField: "movieId",
...       as: "movie"
...     }
...   },
...   {
...     $project: {
...       _id: 0,
...       title: { $first: "$movie.title" },
...       ratings: { $size: "$r" },
...       avgRating: { $round: [{ $avg: "$r.rating" }, 2] }
...     }
...   },
...   { $sort: { ratings: -1 } },
...   { $limit: 10 }
... ])

[
  { title: 'Pulp Fiction (1994)', ratings: 307, avgRating: 4.2 },
  { title: 'The Big Lebowski (1998)', ratings: 106, avgRating: 3.92 },
  { title: 'Clueless (1995)', ratings: 104, avgRating: 3.29 },
  { title: 'Toy Story 2 (1999)', ratings: 97, avgRating: 3.86 },
  { title: 'The Hangover (2009)', ratings: 76, avgRating: 3.63 },
  { title: 'Django Unchained (2012)', ratings: 71, avgRating: 3.94 },
  {
    title: 'Guardians of the Galaxy (2014)',
    ratings: 59,
    avgRating: 4.05
  },
  { title: 'Zombieland (2009)', ratings: 53, avgRating: 3.88 },
  {
    title: '(500) Days of Summer (2009)',
    ratings: 42,
    avgRating: 3.67
  },
  {
    title: 'Home Alone 2: Lost in New York (1992)',
    ratings: 31,
    avgRating: 2.52
  }
]

cinematch> 
```

Result: The movies tagged as funny are listed with their number of ratings and average rating. Pulp Fiction has the highest average (4.20).

48. Most active taggers

Query:

```
db.tags.aggregate([
  {
    $group: {
      _id: "$userId",
      tags: { $sum: 1 },
    },
  },
])
```

```

        movies: { $addToSet: "$movieId" }
    }
},
{
    $project: {
        userId: "$_id",
        _id: 0,
        tags: 1,
        moviesTagged: { $size: "$movies" }
    }
},
{ $sort: { tags: -1 } },
{ $limit: 5 }
])

```

Output:

mongosh - cinematch

```

cinematch> db.tags.aggregate([
...     {
...         $group: {
...             _id: "$userId",
...             tags: { $sum: 1 },
...             movies: { $addToSet: "$movieId" }
...         }
...     },
...     {
...         $project: {
...             userId: "$_id",
...             _id: 0,
...             tags: 1,
...             moviesTagged: { $size: "$movies" }
...         }
...     },
...     { $sort: { tags: -1 } },
...     { $limit: 5 }
... ])

[
  { tags: 1507, userId: 474, moviesTagged: 1235 },
  { tags: 432, userId: 567, moviesTagged: 109 },
  { tags: 370, userId: 62, moviesTagged: 69 },
  { tags: 323, userId: 599, moviesTagged: 5 },
  { tags: 280, userId: 477, moviesTagged: 68 }
]

cinematch>

```

Result: User 474 has given 1,507 tags, which is 41% of all the tags.

Recommendation Queries

49. Content-based: movies most similar to Toy Story by genre overlap

Query:

```

db.movies.aggregate([
  { $match: { movieId: { $ne: 1 } } },
  {
    $addFields: {
      sharedGenres: {
        $size: {
          $setIntersection: [
            "$genreList",
            [

```

```

        "Adventure",
        "Animation",
        "Children",
        "Comedy",
        "Fantasy"
    ]
}
}
}
},
{ $match: { sharedGenres: { $gte: 4 } } },
{
    $lookup: {
        from: "ratings",
        localField: "movieId",
        foreignField: "movieId",
        as: "r"
    }
},
{
    $addFields: {
        ratingCount: { $size: "$r" },
        avgRating: { $avg: "$r.rating" }
    }
},
{ $match: { ratingCount: { $gte: 20 } } },
{ $sort: { sharedGenres: -1, avgRating: -1 } },
{ $limit: 10 },
{
    $project: {
        _id: 0,
        title: 1,
        sharedGenres: 1,
        ratingCount: 1,
        avgRating: { $round: ["$avgRating", 2] }
    }
}
])

```

Output:

```
mongosh - cinematch

cinematch> db.movies.aggregate([
...   { $match: { movieId: { $ne: 1 } } },
...   {
...     $addFields: {
...       sharedGenres: {
...         $size: {
...           $setIntersection: [
...             "$genreList",
...             [
...               "Adventure",
...               "Animation",
...               "Children",
...               "Comedy",
...               "Fantasy"
...             ]
...           }
...         }
...       }
...     },
...     { $match: { sharedGenres: { $gte: 4 } } },
...     {
...       $lookup: {
...         from: "ratings",
...         localField: "movieId",
...         foreignField: "movieId",
...         as: "r"
...       }
...     },
...     {
...       $addFields: {
...         ratingCount: { $size: "$r" },
...         avgRating: { $avg: "$r.rating" }
...       }
...     },
...     { $match: { ratingCount: { $gte: 20 } } },
...     { $sort: { sharedGenres: -1, avgRating: -1 } },
...     { $limit: 10 },
...     {
...       $project: {
...         _id: 0,
...         title: 1,
...         sharedGenres: 1,
...         ratingCount: 1,
...         avgRating: { $round: ["$avgRating", 2] }
...       }
...     }
...   })
... ]
[
  {
    title: 'Toy Story 3 (2010)',
    sharedGenres: 5,
    ratingCount: 55,
    avgRating: 4.11
  },
  {
    title: 'Monsters, Inc. (2001)',
    sharedGenres: 5,
    ratingCount: 132,
    avgRating: 3.87
  },
  {
    title: 'The Lego Movie (2014)',
    sharedGenres: 5,
    ratingCount: 31,
    avgRating: 3.87
  },
  {
    title: 'Shrek (2001)',
    sharedGenres: 5,
    ratingCount: 170,
    avgRating: 3.87
  },
  {
    title: 'Toy Story 2 (1999)',
    sharedGenres: 5,
    ratingCount: 97,
    avgRating: 3.86
  },
  {
    title: 'Inside Out (2015)',
    sharedGenres: 5,
    ratingCount: 43,
    avgRating: 3.81
  },
  {
    title: 'The Emperor's New Groove (2000)',
    sharedGenres: 5,
    ratingCount: 37,
    avgRating: 3.72
  },
  {
    title: 'Who Framed Roger Rabbit? (1988)',
    sharedGenres: 5,
    ratingCount: 97,
    avgRating: 3.57
  },
  {
    title: 'Antz (1998)',
    sharedGenres: 5,
    ratingCount: 45,
    avgRating: 3.24
  },
  {
    title: 'Shrek the Third (2007)',
    sharedGenres: 5,
    ratingCount: 21,
    avgRating: 3.02
  }
]
cinematch> █
```

Result: Toy Story 3, Monsters Inc., The Lego Movie, Shrek and Toy Story 2 share all five genres with Toy Story and are recommended.

50. Collaborative: 'users who liked The Matrix also liked...'

Query:

```
db.ratings.aggregate([
  { $match: { movieId: 2571, rating: { $gte: 4.5 } } },
  { $group: { _id: null, fans: { $addToSet: "$userId" } } },
  {
    $lookup: {
      from: "ratings",
      let: { fans: "$fans" },
      pipeline: [
        {
          $match: {
            $expr: {
              $and: [
                { $in: ["$userId", "$$fans"] },
                { $gte: ["$rating", 4.5] },
                { $ne: ["$movieId", 2571] }
              ]
            }
          }
        },
        { $group: { _id: "$movieId", fanCount: { $sum: 1 } } },
        { $sort: { fanCount: -1 } },
        { $limit: 10 }
      ],
      as: "alsoLiked"
    }
  },
  { $unwind: "$alsoLiked" },
  {
    $lookup: {
      from: "movies",
      localField: "alsoLiked._id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  {
    $project: {
      _id: 0,
      title: { $first: "$movie.title" },
      fansWhoLikedIt: "$alsoLiked.fanCount",
      totalMatrixFans: { $size: "$fans" }
    }
  }
])
```

Output:

```
mongosh - cinematch

cinematch> db.ratings.aggregate([
...   { $match: { movieId: 2571, rating: { $gte: 4.5 } } },
...   { $group: { _id: null, fans: { $addToSet: "$userId" } } },
...   {
...     $lookup: {
...       from: "ratings",
...       let: { fans: "$fans" },
...       pipeline: [
...         {
...           $match: {
...             $expr: {
...               $and: [
...                 { $in: [ "$userId", "$$fans" ] },
...                 { $gte: [ "$rating", 4.5 ] },
...                 { $ne: [ "$movieId", 2571 ] }
...               ]
...             }
...           }
...         },
...         { $group: { _id: "$movieId", fanCount: { $sum: 1 } } },
...         { $sort: { fanCount: -1 } },
...         { $limit: 10 }
...       ],
...       as: "alsoLiked"
...     },
...     { $unwind: "$alsoLiked" },
...     {
...       $lookup: {
...         from: "movies",
...         localField: "alsoLiked._id",
...         foreignField: "movieId",
...         as: "movie"
...       }
...     },
...     {
...       $project: {
...         _id: 0,
...         title: { $first: "$movie.title" },
...         fansWhoLikedIt: "$alsoLiked.fanCount",
...         totalMatrixFans: { $size: "$fans" }
...       }
...     }
...   ]
... })

[
  {
    title: 'Fight Club (1999)',
    fansWhoLikedIt: 75,
    totalMatrixFans: 150
  },
  {
    title: 'The Shawshank Redemption (1994)',
    fansWhoLikedIt: 75,
    totalMatrixFans: 150
  },
  {
    title: 'Pulp Fiction (1994)',
    fansWhoLikedIt: 61,
    totalMatrixFans: 150
  },
  {
    title: 'Star Wars: Episode IV - A New Hope (1977)',
    fansWhoLikedIt: 60,
    totalMatrixFans: 150
  },
  {
    title: 'Star Wars: Episode V - The Empire Strikes Back (1980)',
    fansWhoLikedIt: 59,
    totalMatrixFans: 150
  },
  {
    title: 'Forrest Gump (1994)',
    fansWhoLikedIt: 57,
    totalMatrixFans: 150
  },
  {
    title: 'The Lord of the Rings: The Fellowship of the Ring (2001)',
    fansWhoLikedIt: 56,
    totalMatrixFans: 150
  },
  {
    title: 'The Silence of the Lambs (1991)',
    fansWhoLikedIt: 51,
    totalMatrixFans: 150
  },
  {
    title: 'Star Wars: Episode VI - Return of the Jedi (1983)',
    fansWhoLikedIt: 50,
    totalMatrixFans: 150
  },
  {
    title: 'Saving Private Ryan (1998)',
    fansWhoLikedIt: 49,
    totalMatrixFans: 150
  }
]

cinematch> 
```

Result: Out of 150 users who gave The Matrix 4.5 or 5 stars, 75 also gave high ratings to The Shawshank Redemption and Fight Club.

51. Personalised recommendations for user 1

Query:

```
db.movies.aggregate([
  { $match: { genreList: { $all: ["Action", "Adventure"] } } },
  {
    $lookup: {
      from: "ratings",
      localField: "movieId",
      foreignField: "movieId",
      as: "r"
    }
  },
  { $match: { "r.userId": { $ne: 1 } } },
  {
    $addFields: {
      ratingCount: { $size: "$r" },
      avgRating: { $avg: "$r.rating" }
    }
  },
  { $match: { ratingCount: { $gte: 50 } } },
  { $sort: { avgRating: -1 } },
  { $limit: 10 },
  {
    $project: {
      _id: 0,
      title: 1,
      year: 1,
      ratingCount: 1,
      avgRating: { $round: ["$avgRating", 2] }
    }
  }
])
```

Output:

```
mongosh - cinematch

cinematch> db.movies.aggregate([
...   { $match: { genreList: { $all: ["Action", "Adventure"] } } },
...   {
...     $lookup: {
...       from: "ratings",
...       localField: "movieId",
...       foreignField: "movieId",
...       as: "r"
...     },
...   },
...   { $match: { "r.userId": { $ne: 1 } } },
...   {
...     $addFields: {
...       ratingCount: { $size: "$r" },
...       avgRating: { $avg: "$r.rating" }
...     },
...   },
...   { $match: { ratingCount: { $gte: 50 } } },
...   { $sort: { avgRating: -1 } },
...   { $limit: 10 },
...   {
...     $project: {
...       _id: 0,
...       title: 1,
...       year: 1,
...       ratingCount: 1,
...       avgRating: { $round: ["$avgRating", 2] }
...     }
...   }
... ])

[
  {
    title: 'North by Northwest (1959)',
    year: 1959,
    ratingCount: 57,
    avgRating: 4.18
  },
  {
    title: 'City of God (Cidade de Deus) (2002)',
    year: 2002,
    ratingCount: 75,
    avgRating: 4.15
  },
  {
    title: 'The Good, the Bad and the Ugly (Buono, il brutto, il cattivo, Il) (1966)',
    year: 1966,
    ratingCount: 72,
    avgRating: 4.15
  },
  {
    title: 'The Lord of the Rings: The Return of the King (2003)',
    year: 2003,
    ratingCount: 185,
    avgRating: 4.12
  },
  {
    title: 'Guardians of the Galaxy (2014)',
    year: 2014,
    ratingCount: 59,
    avgRating: 4.05
  },
  {
    title: 'Army of Darkness (1993)',
    year: 1993,
    ratingCount: 51,
    avgRating: 4.04
  },
  {
    title: 'The Dark Knight Rises (2012)',
    year: 2012,
    ratingCount: 76,
    avgRating: 3.99
  },
  {
    title: 'Aliens (1986)',
    year: 1986,
    ratingCount: 126,
    avgRating: 3.96
  },
  {
    title: 'Children of Men (2006)',
    year: 2006,
    ratingCount: 64,
    avgRating: 3.95
  },
  {
    title: 'Casino Royale (2006)',
    year: 2006,
    ratingCount: 81,
    avgRating: 3.94
  }
]

cinematch> 
```

Result: Action and Adventure movies that user 1 has not rated are recommended, such as North by Northwest and City of God.

52. Hidden gems: highly rated but rarely watched

Query:

```
db.ratings.aggregate([
  {
    $group: {
      _id: "$movieId",
      ratingCount: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },
  {
    $match: { ratingCount: { $gte: 10, $lte: 30 }, avgRating: { $gte: 4.3 } }
  },
  {
    $sort: { avgRating: -1 } },
  { $limit: 10 },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  { $unwind: "$movie" },
  {
    $project: {
      _id: 0,
      title: "$movie.title",
      genres: "$movie.genres",
      ratingCount: 1,
      avgRating: { $round: ["$avgRating", 2] }
    }
  }
])
```

Output:

```
mongosh - cinematch

cinematch> db.ratings.aggregate([
...   {
...     $group: {
...       _id: "$movieId",
...       ratingCount: { $sum: 1 },
...       avgRating: { $avg: "$rating" }
...     }
...   },
...   {
...     $match: { ratingCount: { $gte: 10, $lte: 30 }, avgRating: { $gte: 4.3 } }
...   },
...   { $sort: { avgRating: -1 } },
...   { $limit: 10 },
...   {
...     $lookup: {
...       from: "movies",
...       localField: "_id",
...       foreignField: "movieId",
...       as: "movie"
...     }
...   },
...   { $unwind: "$movie" },
...   {
...     $project: {
...       _id: 0,
...       title: "$movie.title",
...       genres: "$movie.genres",
...       ratingCount: 1,
...       avgRating: { $round: [ "$avgRating", 2 ] }
...     }
...   }
... ])

[
  {
    ratingCount: 11,
    title: 'Secrets & Lies (1996)',
    genres: 'Drama',
    avgRating: 4.59
  },
  {
    ratingCount: 11,
    title: 'Guess Who's Coming to Dinner (1967)',
    genres: 'Drama',
    avgRating: 4.55
  },
  {
    ratingCount: 12,
    title: 'Paths of Glory (1957)',
    genres: 'Drama|War',
    avgRating: 4.54
  },
  {
    ratingCount: 20,
    title: 'A Streetcar Named Desire (1951)',
    genres: 'Drama',
    avgRating: 4.47
  },
  {
    ratingCount: 12,
    title: 'The Celebration (Festen) (1998)',
    genres: 'Drama',
    avgRating: 4.46
  },
  {
    ratingCount: 15,
    title: 'Ran (1985)',
    genres: 'Drama|War',
    avgRating: 4.43
  },
  {
    ratingCount: 14,
    title: 'His Girl Friday (1940)',
    genres: 'Comedy|Romance',
    avgRating: 4.39
  },
  {
    ratingCount: 10,
    title: 'All Quiet on the Western Front (1930)',
    genres: 'Action|Drama|War',
    avgRating: 4.35
  },
  {
    ratingCount: 27,
    title: 'Sunset Blvd. (a.k.a. Sunset Boulevard) (1950)',
    genres: 'Drama|Film-Noir|Romance',
    avgRating: 4.33
  },
  {
    ratingCount: 18,
    title: 'The Hustler (1961)',
    genres: 'Drama',
    avgRating: 4.33
  }
]

cinematch> 
```

Result: Movies with a high average but only 10 to 30 ratings are listed, such as Secrets & Lies (4.59) and Paths of Glory (4.54).

53. Materialise per-movie statistics with \$merge

Query:

```
db.ratings.aggregate([
  {
    $group: {
      _id: "$movieId",
      ratingCount: { $sum: 1 },
      avgRating: { $avg: "$rating" },
      stdDev: { $stdDevPop: "$rating" }
    }
  },
  { $merge: { into: "movie_stats", whenMatched: "replace" } }
]);
db.movie_stats.find().sort({ ratingCount: -1 }).limit(5)
```

Output:

mongosh - cinematch

```
cinematch> db.ratings.aggregate([
...   {
...     $group: {
...       _id: "$movieId",
...       ratingCount: { $sum: 1 },
...       avgRating: { $avg: "$rating" },
...       stdDev: { $stdDevPop: "$rating" }
...     }
...   },
...   { $merge: { into: "movie_stats", whenMatched: "replace" } }
... ]);
... db.movie_stats.find().sort({ ratingCount: -1 }).limit(5)

[
  {
    _id: 356,
    ratingCount: 329,
    avgRating: 4.164133738601824,
    stdDev: 0.829979269004478
  },
  {
    _id: 318,
    ratingCount: 317,
    avgRating: 4.429022082018927,
    stdDev: 0.7118931016352933
  },
  {
    _id: 296,
    ratingCount: 307,
    avgRating: 4.197068403908795,
    stdDev: 0.9504453980131069
  },
  {
    _id: 593,
    ratingCount: 279,
    avgRating: 4.161290322580645,
    stdDev: 0.8524515258270362
  },
  {
    _id: 2571,
    ratingCount: 278,
    avgRating: 4.192446043165468,
    stdDev: 0.9734869031917084
  }
]

cinematch> 
```

Result: Statistics of 9,724 movies were written to the movie_stats collection. The 5 most rated movies are shown.

54. Dashboard summary in a single \$facet query

Query:

```
db.ratings.aggregate([
  {
    $facet: {
      overall: [
        {
          $group: {
            _id: null,
            ratings: { $sum: 1 },
            avg: { $avg: "$rating" }
          }
        },
        {
          $project: { _id: 0, ratings: 1, avg: { $round: ["$avg", 2] } }
        }
      ],
      topMovies: [
        { $group: { _id: "$movieId", n: { $sum: 1 } } },
        { $sort: { n: -1 } },
        { $limit: 3 }
      ],
      topUsers: [
        { $group: { _id: "$userId", n: { $sum: 1 } } },
        { $sort: { n: -1 } },
        { $limit: 3 }
      ],
      byStar: [
        { $group: { _id: "$rating", n: { $sum: 1 } } },
        { $sort: { _id: -1 } }
      ]
    }
  }
])
```

Output:

mongosh - cinematch

```
cinematch> db.ratings.aggregate([
...   {
...     $facet: {
...       overall: [
...         {
...           $group: {
...             _id: null,
...             ratings: { $sum: 1 },
...             avg: { $avg: "$rating" }
...           }
...         },
...         {
...           $project: { _id: 0, ratings: 1, avg: { $round: ["$avg", 2] } }
...         }
...       ],
...       topMovies: [
...         { $group: { _id: "$movieId", n: { $sum: 1 } } },
...         { $sort: { n: -1 } },
...         { $limit: 3 }
...       ],
...       topUsers: [
...         { $group: { _id: "$userId", n: { $sum: 1 } } },
...         { $sort: { n: -1 } },
...         { $limit: 3 }
...       ],
...       byStar: [
...         { $group: { _id: "$rating", n: { $sum: 1 } } },
...         { $sort: { _id: -1 } }
...       ]
...     }
...   }
... ])

[
  {
    overall: [ { ratings: 100836, avg: 3.5 } ],
    topMovies: [
      { _id: 356, n: 329 },
      { _id: 318, n: 317 },
      { _id: 296, n: 307 }
    ],
    topUsers: [
      { _id: 414, n: 2698 },
      { _id: 599, n: 2478 },
      { _id: 474, n: 2108 }
    ],
    byStar: [
      { _id: 5, n: 13211 },
      { _id: 4.5, n: 8551 },
      { _id: 4, n: 26818 },
      { _id: 3.5, n: 13136 },
      { _id: 3, n: 20047 },
      { _id: 2.5, n: 5550 },
      { _id: 2, n: 7551 },
      { _id: 1.5, n: 1791 },
      { _id: 1, n: 2811 },
      { _id: 0.5, n: 1370 }
    ]
  }
]
cinematch> 
```

Result: One \$facet query returns the overall statistics, top movies, top users and rating distribution together.

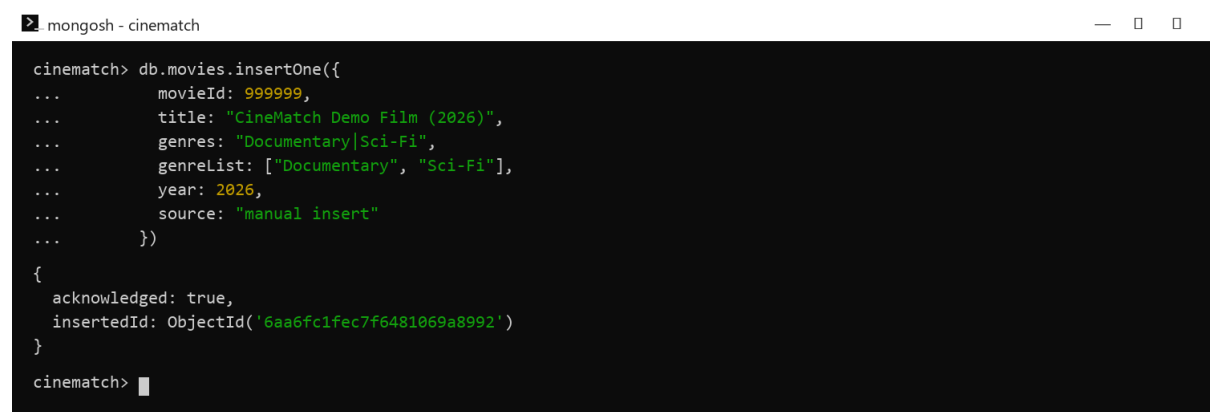
CRUD & Index Performance

55. Insert a new movie document

Query:

```
db.movies.insertOne({
  movieId: 999999,
  title: "CineMatch Demo Film (2026)",
  genres: "Documentary|Sci-Fi",
  genreList: ["Documentary", "Sci-Fi"],
  year: 2026,
  source: "manual insert"
})
```

Output:

A screenshot of a MongoDB terminal window titled 'mongosh - cinematch'. The terminal shows a command to insert a new movie document into the 'movies' collection. The document has fields: movieId (999999), title ('CineMatch Demo Film (2026)'), genres ('Documentary|Sci-Fi'), genreList (an array containing 'Documentary' and 'Sci-Fi'), year (2026), and source ('manual insert'). The output shows that the document was successfully inserted, with 'acknowledged: true' and an 'insertedId' of '6aa6fc1fec7f6481069a8992'.

Result: The new document was inserted even though it has an extra field (source) which other movies do not have.

56. Update the inserted document (\$set and \$push)

Query:

```
db.movies.updateOne(
  { movieId: 999999 },
  { $push: { genreList: "Drama" }, $set: { genres: "Documentary|Sci-Fi|Drama", verified: true } }
);
db.movies.findOne({ movieId: 999999 })
```

Output:

```
mongosh - cinematch

cinematch> db.movies.updateOne(
...     { movieId: 999999 },
...     { $push: { genreList: "Drama" }, $set: { genres: "Documentary|Sci-Fi|Drama", verified: true } }
... );
... db.movies.findOne({ movieId: 999999 })
{
  _id: ObjectId('6aa6fc1fec7f6481069a8992'),
  movieId: 999999,
  title: 'CineMatch Demo Film (2026)',
  genres: 'Documentary|Sci-Fi|Drama',
  genreList: [ 'Documentary', 'Sci-Fi', 'Drama' ],
  year: 2026,
  source: 'manual insert',
  verified: true
}
cinematch>
```

Result: 'Drama' was added to the genre array and the verified field was added using one updateOne.

57. Delete the demo document

Query:

```
db.movies.deleteOne({ movieId: 999999 });
db.movies.countDocuments()
```

Output:

```
mongosh - cinematch

cinematch> db.movies.deleteOne({ movieId: 999999 });
... db.movies.countDocuments()
9742
cinematch>
```

Result: After deleting the demo document the movies collection has 9,742 documents again.

58. Explain plan: index scan vs collection scan

Query:

```
const plan = db.ratings.find({ movieId: 296 }).explain("executionStats");
({
  stage: plan.queryPlanner.winningPlan.stage,
  inputStage: plan.queryPlanner.winningPlan.inputStage?.stage,
  indexName: plan.queryPlanner.winningPlan.inputStage?.indexName,
  docsReturned: plan.executionStats.nReturned,
  docsExamined: plan.executionStats.totalDocsExamined,
  keysExamined: plan.executionStats.totalKeysExamined,
  executionTimeMs: plan.executionStats.executionTimeMillis
})
```

Output:

```
mongosh - cinematch

cinematch> const plan = db.ratings.find({ movieId: 296 }).explain("executionStats");
...
  ({
...
    stage: plan.queryPlanner.winningPlan.stage,
...
    inputStage: plan.queryPlanner.winningPlan.inputStage?.stage,
...
    indexName: plan.queryPlanner.winningPlan.inputStage?.indexName,
...
    docsReturned: plan.executionStats.nReturned,
...
    docsExamined: plan.executionStats.totalDocsExamined,
...
    keysExamined: plan.executionStats.totalKeysExamined,
...
    executionTimeMs: plan.executionStats.executionTimeMillis
...
  })
{
  stage: 'FETCH',
  inputStage: 'IXSCAN',
  indexName: 'movieId_1',
  docsReturned: 307,
  docsExamined: 307,
  keysExamined: 307,
  executionTimeMs: 0
}
cinematch> 
```

Result: The query uses the movieId index (IXSCAN). Only 307 documents were examined instead of all 100,836.

MongoDB Operators Used

Operator	Use in this project
find, findOne, countDocuments, distinct	Reading documents, counting and unique values
\$regex, \$text	Pattern matching and text search on titles
\$all, \$in, \$ne, \$gte, \$lte, \$size, \$expr	Filtering conditions on fields and arrays
insertOne, updateOne, updateMany, deleteOne	CRUD operations
\$set, \$push	Adding fields and array elements in updates
\$match	Filters documents in the pipeline
\$group with \$sum, \$avg, \$min, \$max, \$first, \$addToSet, \$stdDevPop	Grouping and calculating values
\$sort, \$limit, \$count, \$project, \$addFields	Sorting, limiting and shaping the output
\$lookup	Joining information from another collection
\$unwind	Expanding an array into separate documents
\$facet, \$bucket	Multiple results in one query, grouping into ranges
\$split, \$regexFind, \$concat, \$toLower, \$toInt, \$toDate	String and type conversion
\$year, \$dayOfWeek	Date parts
\$setIntersection, \$arrayElemAt, \$slice	Array operations
\$merge	Writing pipeline results into a collection
createIndex, getIndexes, explain	Indexing

CineMatch Web Application

A web application was developed so that the analysis can be viewed without opening mongosh. The backend is written in Node.js with Express and connects to MongoDB using the official driver. It has API endpoints for the overview statistics, movie search, movie details with recommendations, genre analysis, user profiles and for running any of the queries above. The frontend is written in React and shows the results as tables and charts.

Endpoint	Description
GET /api/overview	Counts, rating distribution, ratings per year, genre statistics, top movies
GET /api/movies	Search movies by title, genre and sort order
GET /api/movies/:id	Movie details, rating breakdown, tags and recommendations
GET /api/genres/:genre	Movies per decade and top movies of a genre
GET /api/users/:id	User profile, favourite genres and recommended movies
GET /api/queries/:id/run	Runs one of the queries of this report



Overview page


CineMatch
MONGODB
ANALYTICS

Overview

Explore movies

Genres

Users & recs

Query Lab

EXPLORE

Movie catalogue

Regex title search, multkey-index genre filters and sorting on the pre-aggregated movie_stats collection.

Search

Highest rated ▾

≥ 100 ratings ▾

All genres
Action
Adventure
Animation
Children
Comedy
Crime
Documentary
Drama
Fantasy
Film-Noir
Horror
IMAX
Musical
Mystery
Romance
Sci-Fi
Thriller
War
Western

138 movies

#1

The Shawshank Redemption

1994

★ 4.43 · 317

Crime Drama

#2

The Godfather

1972

★ 4.29 · 192

Crime Drama

#3

Fight Club

1999

★ 4.27 · 218

Action Crime Drama

#4

The Godfather: Part II

1974

★ 4.26 · 129

Crime Drama

#5

The Departed

2006

★ 4.25 · 107

Crime Drama Thriller

#6

Goodfellas

1990

★ 4.25 · 126

Crime Drama

#7

Casablanca

1942

★ 4.24 · 100

Drama Romance

#8

The Dark Knight

2008

★ 4.24 · 149

Action Crime Drama

#9

The Usual Suspects

1995

★ 4.24 · 204

Crime Mystery Thriller

#10

The Princess Bride

1987

★ 4.23 · 142

Action Adventure Comedy

#11

Star Wars: Episode IV - A New Hope

1977

★ 4.23 · 251

Action Adventure Sci-Fi

#12

Schindler's List

1993

★ 4.22 · 220

Drama War

#13

Apocalypse Now

1979

★ 4.22 · 107

Action Drama War

#14

American History X

1998

★ 4.22 · 129

Crime Drama

#15

Star Wars: Episode V - The Empire Strikes Back

1980

★ 4.22 · 211

Action Adventure Sci-Fi

#16

Raiders of the Lost Ark (Indiana Jones and the Raiders of the Lost Ark)

1981

★ 4.21 · 200

Action Adventure

#17

One Flew Over the Cuckoo's Nest

1975

★ 4.20 · 133

Drama

#18

Reservoir Dogs

1992

★ 4.20 · 131

Crime Mystery Thriller

#19

Pulp Fiction

1994

★ 4.20 · 307

Comedy Crime Drama

#20

The Matrix

1999

★ 4.19 · 278

Action Sci-Fi Thriller

#21

#22

#23

#24

Database cinematch

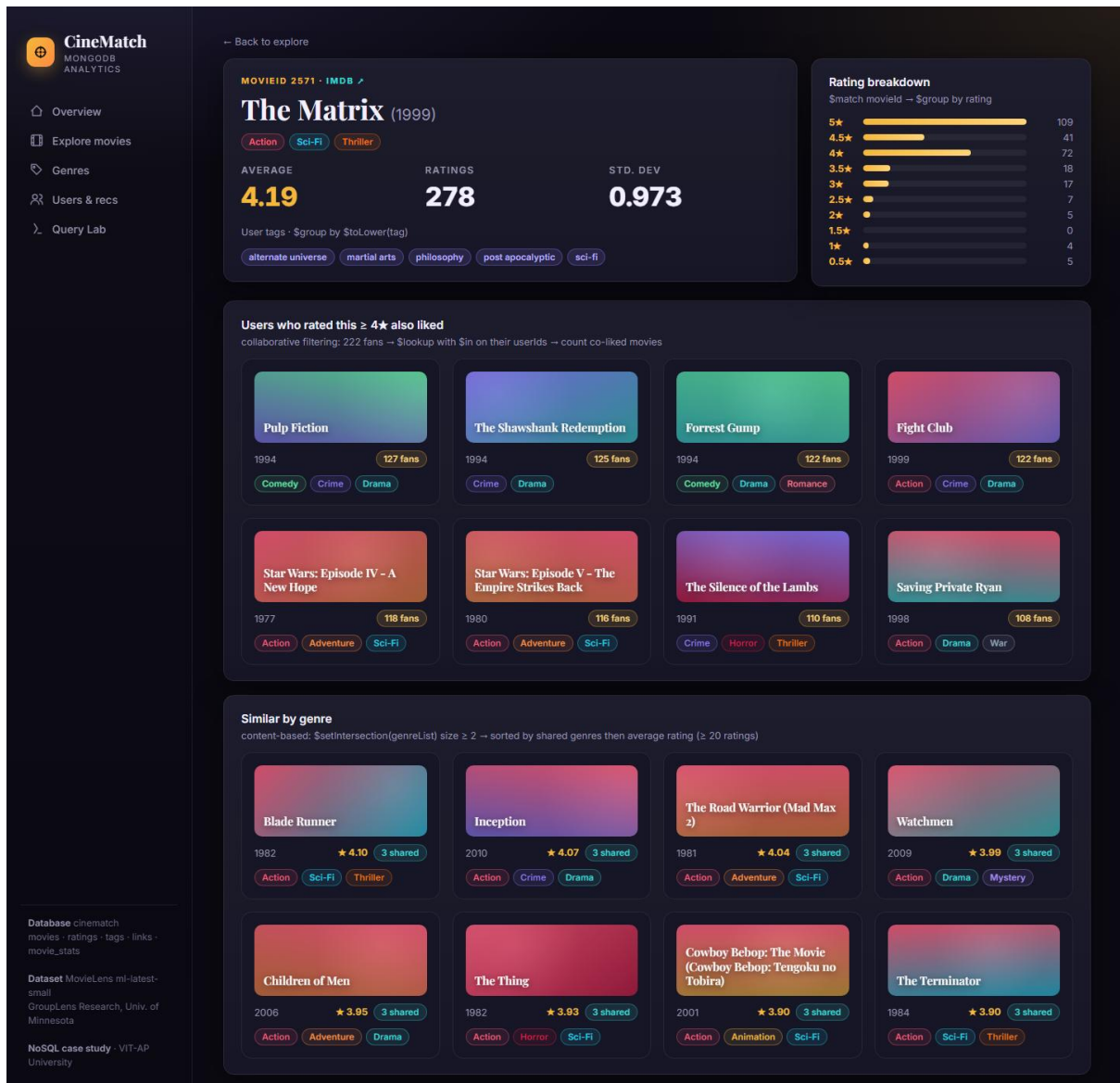
movies · ratings · tags · links · movie_stats

Dataset MovieLens ml-latest-small

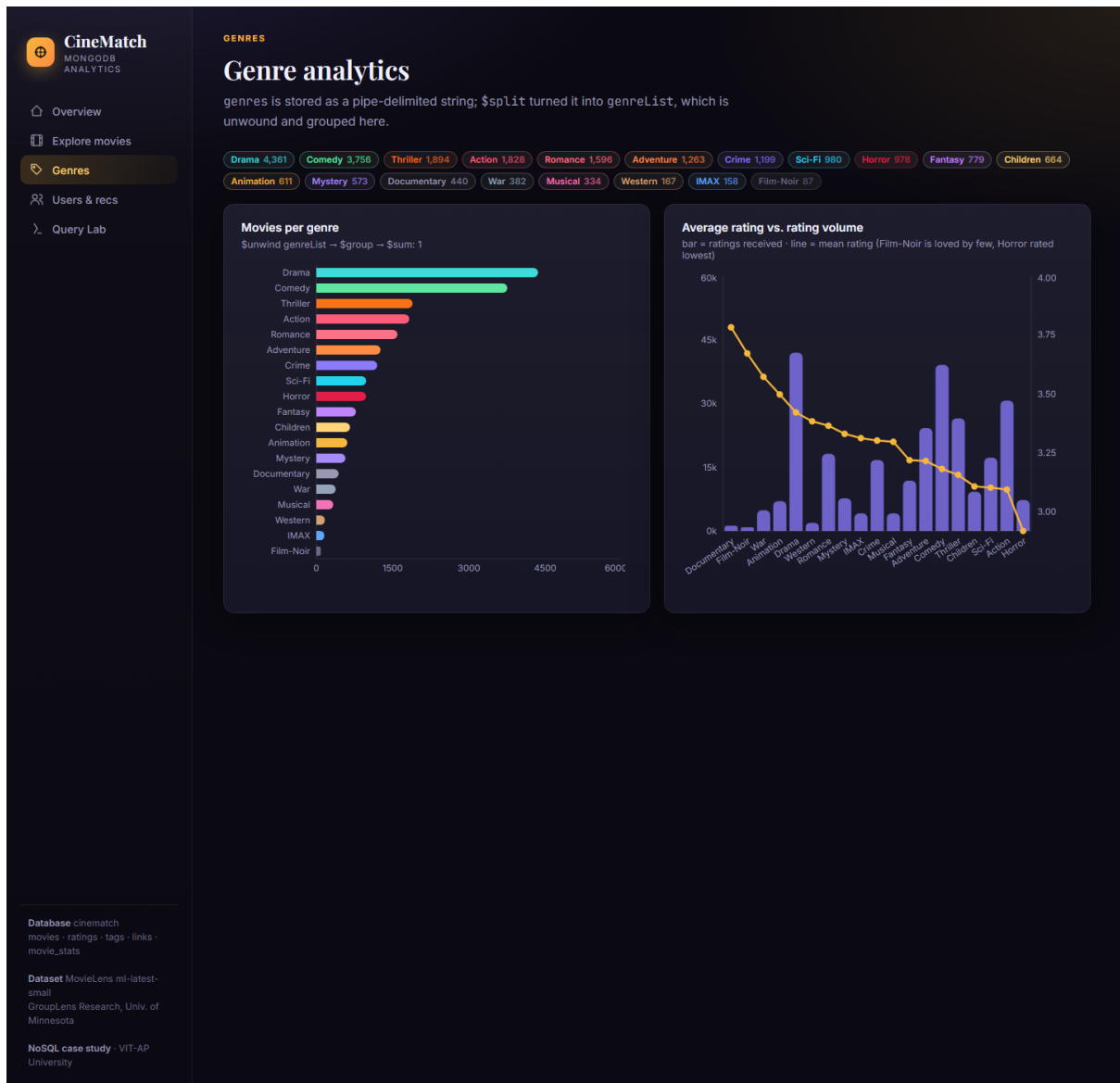
GroupLens Research, Univ. of Minnesota

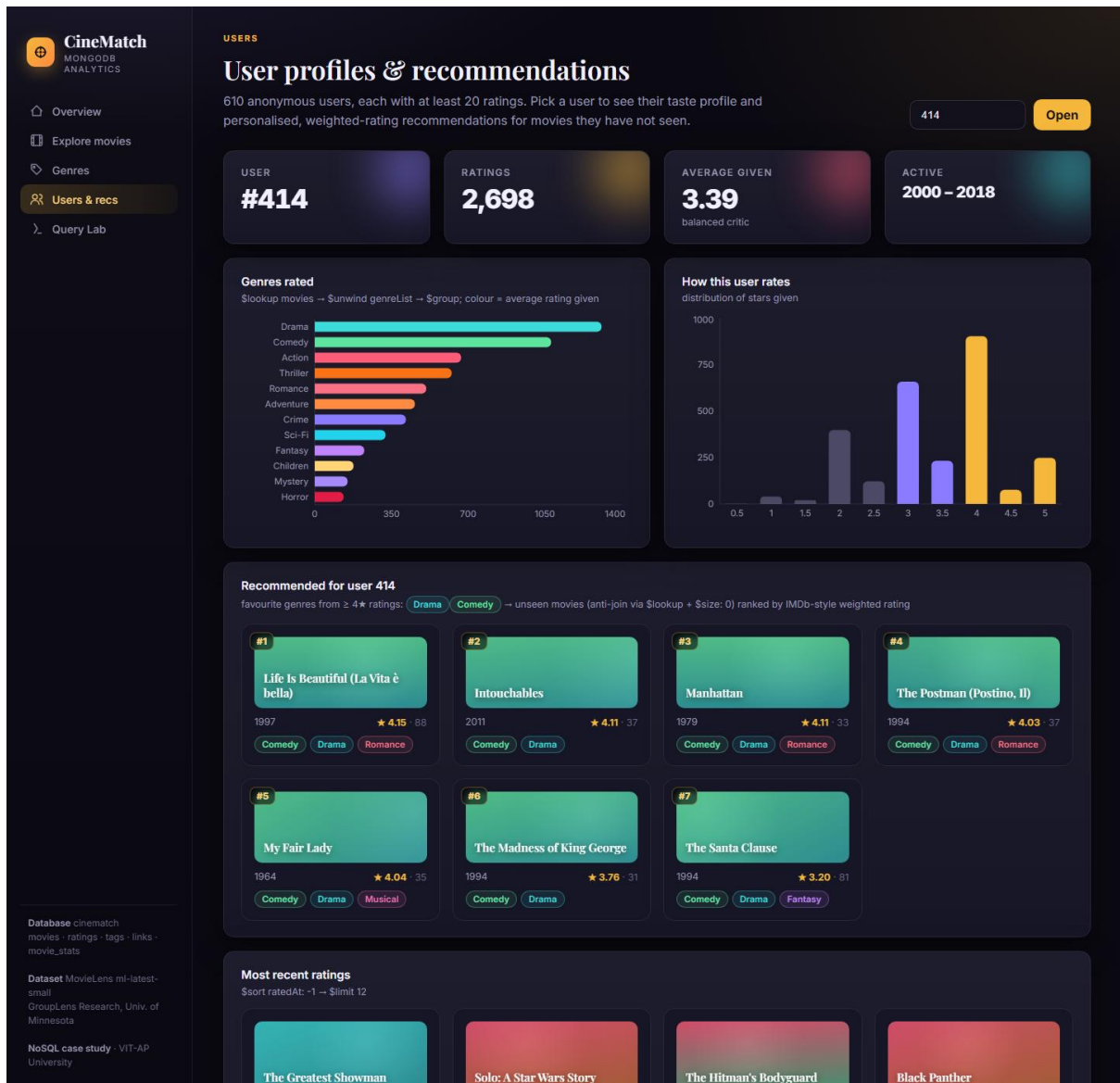
NoSQL case study · VIT-AP University

Explore page - highest rated movies with at least 100 ratings



Movie page of The Matrix with recommendations





User profile page of user 414 with recommendations


CineMatch
MONGODB
ANALYTICS

Overview
Explore movies
Genres
Users & recs
Query Lab

Database: cinematch
collections: movies, ratings, tags, links, movie_stats
Dataset: MovieLens ml-latest-small
GroupLens Research, Univ. of Minnesota
NoSQL case study - VIT-AP University

QUERY LAB

All 57 MongoDB queries

Every query from the case study, exactly as typed in mongosh. Select one to execute it live against the cinematch database and see the result as a table, chart or raw JSON.

DATABASE OVERVIEW

- List all collections in the cinematch database
- Count documents in every collection
- Display a sample movie document
- Display a sample rating document
- Display a sample tag and link document
- Count the number of distinct users who rated movies

DATA CLEANING & SCHEMA ENRICHMENT

- Extract the release year from the movie title
- Split the pipe-delimited genres string into an array
- Convert Unix timestamps into ISODate fields
- Verify the enriched movie document
- Create indexes on frequently queried fields

MOVIE EXPLORATION

- Find movies released in 1995
- Search movie titles with a regular expression
- Find movies that are both Comedy and Romance
- Count movies with no genres listed
- Number of movies per genre
- Number of movies released per decade
- Top 10 years with the most movie releases
- Most common genre combinations
- Full-text search on movie titles

RATING ANALYSIS

- Rating distribution (how often each star value is used)
- Overall rating statistics
- Average rating of a specific movie (Toy Story)
- Top 10 most-rated (most popular) movies

RECOMMENDATION QUERIES - QUERY 50

Personalised recommendations for user 1

Top weighted-rated Action/Adventure movies (user 1's favourite genres) that user 1 has NOT rated yet.

aggregate
db.movies
\$match
\$all
\$lookup
\$ne
\$addFields
\$size
\$avg
\$gte
\$sort
\$limit
\$project
\$round

```

db.movies.aggregate([
  { $match: { genreList: { $all: ["Action", "Adventure"] } } },
  {
    $lookup: {
      from: "ratings",
      localField: "movieId",
      foreignField: "movieId",
      as: "r"
    }
  },
  { $match: { "r.userId": { $ne: 1 } } },
  {
    $addFields: {
      ratingCount: { $size: "$r" },
      avgRating: { $avg: "$r.rating" }
    }
  },
  { $match: { ratingCount: { $gte: 50 } } },
  { $sort: { avgRating: -1 } },
  { $limit: 10 },
  { $project: {

```

Run again
33 ms · 10 documents

Chart
Table
JSON



Movie Title	Avg Rating
North by Northwest (1959)	4.0
City of God (Cidade de Deus) (2002)	4.0
The Good, the Bad and the Ugly (1966)	4.0
The Lord of the Rings: The Return of the King (2003)	4.0
Guardians of the Galaxy (2014)	4.0
Army of Darkness (1993)	4.0
The Dark Knight Rises (2012)	4.0
Aliens (1986)	4.0
Children of Men (2006)	4.0
Casino Royale (2006)	4.0

Query page running Query 50

Conclusion

This project helped us understand how MongoDB can be used to store and analyse movie rating data. The MovieLens data was imported into collections, cleaned inside the database and analysed using find queries and aggregation pipelines. We were able to find the most popular and highest rated movies, the average rating of each genre, the most active users, the most used tags and the rating activity over the years. Using \$lookup, \$unwind, \$facet, \$bucket and \$merge we could answer questions that need data from more than one collection.

We also learnt how recommendations can be made using only database queries, by comparing genres, by finding the other movies liked by the fans of a movie and by using a weighted rating. Creating indexes and checking the query plan with explain() showed how indexes make the queries faster. Finally, connecting MongoDB to a web application showed how the same queries can be used in a real application.

References

11. MongoDB Documentation - MongoDB Manual. <https://www.mongodb.com/docs/manual/>
12. MongoDB Aggregation - MongoDB Documentation.
<https://www.mongodb.com/docs/manual/aggregation/>
13. MongoDB Query and Aggregation Operators.
<https://www.mongodb.com/docs/manual/reference/operator/>
14. MongoDB Indexes. <https://www.mongodb.com/docs/manual/indexes/>
15. MongoDB Shell (mongosh) Documentation. <https://www.mongodb.com/docs/mongodb-shell/>
16. MongoDB Node.js Driver. <https://www.mongodb.com/docs/drivers/node/current/>
17. GroupLens Research - MovieLens Latest Datasets.
<https://grouplens.org/datasets/movielens/latest/>
18. Kaggle - Popular Movies Datasets (9742 Movies).
<https://www.kaggle.com/datasets/whenamancodes/popular-movies-datasets-9000-movies>
19. F. Maxwell Harper and Joseph A. Konstan. The MovieLens Datasets: History and Context. ACM Transactions on Interactive Intelligent Systems, 2015.
20. Express.js Documentation. <https://expressjs.com/>
21. React Documentation. <https://react.dev/>